第2章 线性表

【考纲内容】

（一）线性表的基本概念

（二）线性表的实现

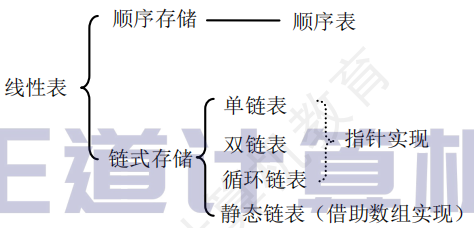
顺序存储；链式存储

（三）线性表的应用

扫一扫

视频讲解

【知识框架】



王道计算机教育
www.cskaoyan.com

【复习提示】

线性表是算法题命题的重点。这类算法题的实现比较容易且代码量较少，但是要求具有最优的性能（时间/空间复杂度），才能获得满分。因此，应牢固掌握线性表的各种基本操作（基于两种存储结构），在平时的学习中多注重培养动手能力。另需提醒的是，算法最重要的是思想！考场上的时间紧迫，在试卷上不一定要求代码具有实际的可执行性，因此应尽力表达出算法的思想和步骤，而不必过于拘泥所有细节。此外，采用时间/空间复杂度较差的方法也能拿到大部分分数，因此在时间紧迫的情况下，建议直接采用暴力法。注意，算法题只能用C/C++语言实现。

2.1 线性表的定义和基本操作

2.1.1 线性表的定义

线性表是具有相同数据类型的 n（$n \geqslant 0$）个数据元素的有限序列，其中 n 为表长，当 $n = 0$ 时线性表是一个空表。若用 L 命名线性表，则其一般表示为

$$L = (a_1, a_2, \cdots, a_i, a_{i+1}, \cdots, a_n)$$

式中，a_1 是唯一的“第一个”数据元素，也称表头元素；a_n 是唯一的“最后一个”数据元素，也称表尾元素。除第一个元素外，每个元素有且仅有一个直接前驱。除最后一个元素外，每个元素有且仅有一个直接后继（“直接前驱”和“前驱”、“直接后继”和“后继”通常被视为同义词）。以上就是线性表的逻辑特性，这种线性有序的逻辑结构正是线性表名字的由来。

配套课程公众号[考机研]
永久联系微信:4550060

由此，我们得出线性表的特点如下：

- 表中元素的个数有限。
- 表中元素具有逻辑上的顺序性，表中元素有其先后次序。
- 表中元素都是数据元素，每个元素都是单个元素。
- 表中元素的数据类型都相同，这意味着每个元素占有相同大小的存储空间。
- 表中元素具有抽象性，即仅讨论元素间的逻辑关系，而不考虑元素究竟表示什么内容。

注　意

线性表是一种逻辑结构，表示元素之间一对一的相邻关系。顺序表和链表是指存储结构，两者属于不同层面的概念，因此不要将其混淆。

2.1.2　线性表的基本操作

一个数据结构的基本操作是指其最核心、最基本的操作。其他较复杂的操作可通过调用其基本操作来实现。线性表的主要操作如下。

- `InitList(&L)`：初始化表。构造一个空的线性表。
- `Length(L)`：求表长。返回线性表 `L` 的长度，即 `L` 中数据元素的个数。
- `LocateElem(L,e)`：按值查找操作。在表 `L` 中查找具有给定关键字值的元素。
- `GetElem(L,i)`：按位查找操作。获取表 `L` 中第 `i` 个位置的元素的值。
- `ListInsert(&L,i,e)`：插入操作。在表 `L` 中的第 `i` 个位置上插入指定元素 `e`。
- `ListDelete(&L,i,&e)`：删除操作。删除表 `L` 中第 `i` 个位置的元素，并用 `e` 返回删除元素的值。
- `PrintList(L)`：输出操作。按前后顺序输出线性表 `L` 的所有元素值。
- `Empty(L)`：判空操作。若 `L` 为空表，则返回 `true`，否则返回 `false`。
- `DestroyList(&L)`：销毁操作。销毁线性表，并释放线性表 `L` 所占用的内存空间。

注　意

①基本操作的实现取决于采用哪种存储结构，存储结构不同，算法的实现也不同。②符号“&”表示 C++语言中的引用调用，在 C 语言中采用指针也可达到同样的效果。

2.1.3　本节试题精选

单项选择题

01. 线性表是具有 n 个（　）的有限序列。

A. 数据表　　B. 字符　　C. 数据元素　　D. 数据项

02. 下列几种描述中，（　）是一个线性表。

A. 由 n 个实数组成的集合　　B. 由 100 个字符组成的序列

C. 所有整数组成的序列　　D. 邻接表

03. 在线性表中，除开始元素外，每个元素（　）。

A. 只有唯一的前驱元素　　B. 只有唯一的后继元素

C. 有多个前驱元素　　D. 有多个后继元素

04. 若非空线性表中的元素既没有直接前驱，又没有直接后继，则该表中有（　）个元素。

A. 1　　B. 2　　C. 3　　D. n

配套课程公众号[考机研]
永久联系微信:4550060

2.1.4 答案与解析

单项选择题

01. C

线性表是由具有相同数据类型的有限数据元素组成的，数据元素是由数据项组成的。

02. B

线性表定义的要求为：相同数据类型、有限序列。选项C的元素个数是无穷个，错误；选项A集合中的元素没有前后驱关系，错误；选项D属于一种存储结构，本题要求选出的是一个具体的线性表，不要将二者混为一谈。只有选项B符合线性表定义的要求。

03. A

线性表中，除最后一个（或第一个）元素外，每个元素都只有一个后继（或前驱）元素。

04. A

线性表中的第一个元素没有直接前驱，最后一个元素没有直接后继；当线性表中仅有一个元素时，该元素既没有直接前驱，又没有直接后继。

2.2 线性表的顺序表示

2.2.1 顺序表的定义

命题追踪 ▶▶（算法题）顺序表的应用（2010、2011、2018、2020）

线性表的顺序存储也称顺序表。它是用一组地址连续的存储单元依次存储线性表中的数据元素，从而使得逻辑上相邻的两个元素在物理位置上也相邻。第1个元素存储在顺序表的起始位置，第 i 个元素的存储位置后面紧接着存储的是第 $i+1$ 个元素，称 i 为元素 a_i 在顺序表中的位序。因此，顺序表的特点是表中元素的逻辑顺序与其存储的物理顺序相同。

假设顺序表 L 存储的起始位置为 LOC(A)，sizeof(ElemType)是每个数据元素所占用存储空间的大小，则表 L 所对应的顺序存储结构如图2.1所示。

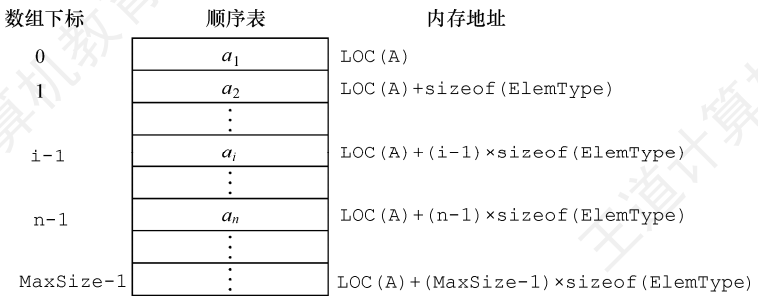


图2.1 线性表的顺序存储结构

每个数据元素的存储位置都和顺序表的起始位置相差一个和该数据元素的位序成正比的常数，因此，顺序表中的任意一个数据元素都可以随机存取，所以线性表的顺序存储结构是一种随机存取的存储结构。通常用高级程序设计语言中的数组来描述线性表的顺序存储结构。

注 意

线性表中元素的位序是从1开始的，而数组中元素的下标是从0开始的。

配套课程公众号[考机研]
永久联系微信:4550060

假定线性表的元素类型为 ElemType，则静态分配的顺序表存储结构描述为

```
#define MaxSize 50                  //定义线性表的最大长度
typedef struct{
    ElemType data[MaxSize];         //顺序表的元素
    int length;                     //顺序表的当前长度
}SqList;                            //顺序表的类型定义
```

一维数组可以是静态分配的，也可以是动态分配的。对数组进行静态分配时，因为数组的大小和空间事先已经固定，所以一旦空间占满，再加入新数据就会产生溢出，进而导致程序崩溃。

而在动态分配时，存储数组的空间是在程序执行过程中通过动态存储分配语句分配的，一旦数据空间占满，就另外开辟一块更大的存储空间，将原表中的元素全部拷贝到新空间，从而达到扩充数组存储空间的目的，而不需要为线性表一次性地划分所有空间。

动态分配的顺序表存储结构描述为

```
#define InitSize 100                //表长度的初始定义
typedef struct{
    ElemType *data;                 //指示动态分配数组的指针
    int MaxSize,length;             //数组的最大容量和当前个数
}SeqList;                           //动态分配数组顺序表的类型定义
```

C 的初始动态分配语句为

```
L.data=(ElemType*)malloc(sizeof(ElemType)*InitSize);
```

C++的初始动态分配语句为

```
L.data=new ElemType[InitSize];
```

注　意

动态分配并不是链式存储，它同样属于顺序存储结构，物理结构没有变化，依然是随机存取方式，只是分配的空间大小可以在运行时动态决定。

王道计算机教育 www.cskaoyan.com

顺序表的主要优点：①可进行随机访问，即可通过首地址和元素序号可以在 $O(1)$时间内找到指定的元素；②存储密度高，每个结点只存储数据元素。顺序表的缺点也很明显：①元素的插入和删除需要移动大量的元素，插入操作平均需要移动 $n/2$ 个元素，删除操作平均需要移动 $(n-1)/2$ 个元素；②顺序存储分配需要一段连续的存储空间，不够灵活。

2.2.2　顺序表上基本操作的实现

命题追踪 ▶ 顺序表上操作的时间复杂度分析（2023）

这里仅讨论顺序表的初始化、插入、删除和按值查找，其他基本操作的算法都很简单。

注　意

在各种操作的实现中（包括严蔚敏老师撰写的教材），往往可以忽略边界条件判断、变量定义、内存分配不足等细节，即不要求代码具有可执行性，而重点在于算法的思想。

1．顺序表的初始化

静态分配和动态分配的顺序表的初始化操作是不同的。静态分配在声明一个顺序表时，就已为其分配了数组空间，因此初始化时只需将顺序表的当前长度设为 0。

```
//SqList L;                         //声明一个顺序表
void InitList(SqList &L){
    L.length=0;                     //顺序表初始长度为 0
}
```

配套课程公众号[考机研]
永久联系微信:4550060

动态分配的初始化为顺序表分配一个预定义大小的数组空间，并将顺序表的当前长度设为0。MaxSize指示顺序表当前分配的存储空间大小，一旦因插入元素而空间不足，就进行再分配。

```
void InitList(SeqList &L){
    L.data=(ElemType *)malloc(InitSize*sizeof(ElemType)); //分配存储空间
    L.length=0;                    //顺序表初始长度为0
    L.MaxSize=InitSize;            //初始存储容量
}
```

2．插入操作

在顺序表L的第i（1<=i<=L.length+1）个位置插入新元素e。若i的输入不合法，则返回false，表示插入失败；否则，将第i个元素及其后的所有元素依次往后移动一个位置，腾出一个空位置插入新元素e，顺序表长度增加1，插入成功，返回true。

```
bool ListInsert(SqList &L,int i,ElemType e){
    if(i<1||i>L.length+1)          //判断i的范围是否有效
        return  false;
    if(L.length>=MaxSize)          //当前存储空间已满，不能插入
        return  false;
    for(int j=L.length;j>=i;j--)   //将第i个元素及之后的元素后移
        L.data[j]=L.data[j-1];
    L.data[i-1]=e;                 //在位置i处放入e
    L.length++;                    //线性表长度加1
    return  true;
}
```

注　意

区别顺序表的位序和数组下标。为何判断插入位置是否合法时if语句中用length+1，而移动元素的for语句中只用length?

最好情况：在表尾插入（$i=n+1$），元素后移语句将不执行，时间复杂度为$O(1)$。

最坏情况：在表头插入（$i=1$），元素后移语句将执行n次，时间复杂度为$O(n)$。

平均情况：假设p_i（$p_i=1/(n+1)$）是在第i个位置上插入一个结点的概率，则在长度为n的线性表中插入一个结点时，所需移动结点的平均次数为

$$\sum_{i=1}^{n+1}p_i(n-i+1)=\sum_{i=1}^{n+1}\frac{1}{n+1}(n-i+1)=\frac{1}{n+1}\sum_{i=1}^{n+1}(n-i+1)=\frac{1}{n+1}\frac{n(n+1)}{2}=\frac{n}{2}$$

因此，顺序表插入算法的平均时间复杂度为$O(n)$。

3．删除操作

删除顺序表L中第i（1<=i<=L.length）个位置的元素，用引用变量e返回。若i的输入不合法，则返回false；否则，将被删元素赋给引用变量e，并将第i+1个元素及其后的所有元素依次往前移动一个位置，返回true。

```
bool ListDelete(SqList &L,int i,ElemType &e){
    if(i<1||i>L.length)            //判断i的范围是否有效
        return  false;
     e=L.data[i-1];                //将被删除的元素赋值给e
    for(int j=i;j<L.length;j++)    //将第i个位置后的元素前移
        L.data[j-1]=L.data[j];
    L.length--;                    //线性表长度减1
    return  true;
}
```

最好情况：删除表尾元素（$i=n$），无须移动元素，时间复杂度为$O(1)$。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

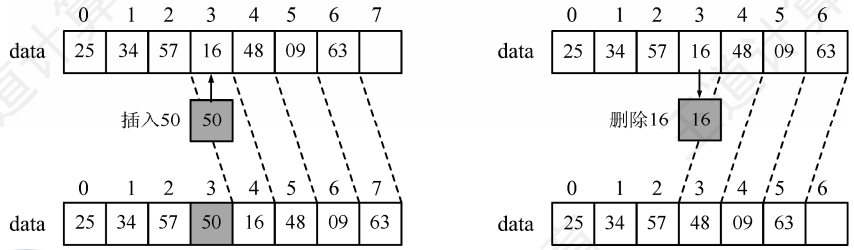
最坏情况：删除表头元素（$i=1$），需移动除表头元素外的所有元素，时间复杂度为 $O(n)$。

平均情况：假设 p_i（$p_i=1/n$）是删除第 i 个位置上结点的概率，则在长度为 n 的线性表中删除一个结点时，所需移动结点的平均次数为

$$\sum_{i=1}^{n} p_i(n-i)=\sum_{i=1}^{n}\frac{1}{n}(n-i)=\frac{1}{n}\sum_{i=1}^{n}(n-i)=\frac{1}{n}\frac{n(n-1)}{2}=\frac{n-1}{2}$$

因此，顺序表删除算法的平均时间复杂度为 $O(n)$。

可见，顺序表中插入和删除操作的时间主要耗费在移动元素上，而移动元素的个数取决于插入和删除元素的位置。图 2.2 所示为一个顺序表在进行插入和删除操作前、后的状态，以及其数据元素在存储空间中的位置变化和表长变化。在图 2.2(a)中，将第 4 个至第 7 个元素从后往前依次后移一个位置，在图 2.2(b)中，将第 5 个至第 7 个元素从前往后依次前移一个位置。



(a) 插入新元素示例　　(b) 删除表中元素示例

图 2.2　顺序表的插入和删除

4. 按值查找（顺序查找）

在顺序表 L 中查找第一个元素值等于 e 的元素，并返回其位序。

```
int LocateElem(SqList L,ElemType e){
    int i;
    for(i=0;i<L.length;i++)
        if(L.data[i]==e)
            return  i+1;        //下标为 i 的元素值等于 e，返回其位序 i+1
    return  0;                  //退出循环，说明查找失败
}
```

最好情况：查找的元素就在表头，仅需比较一次，时间复杂度为 $O(1)$。

最坏情况：查找的元素在表尾（或不存在）时，需要比较 n 次，时间复杂度为 $O(n)$。

平均情况：假设 p_i（$p_i=1/n$）是查找的元素在第 i（1<=i<=L.length）个位置上的概率，则在长度为 n 的线性表中查找值为 e 的元素所需比较的平均次数为

$$\sum_{i=1}^{n} p_i\cdot i=\sum_{i=1}^{n}\frac{1}{n}\cdot i=\frac{1}{n}\frac{n(n+1)}{2}=\frac{n+1}{2}$$

因此，顺序表按值查找算法的平均时间复杂度为 $O(n)$。

顺序表的按序号查找非常简单，即直接根据数组下标访问数组元素，其时间复杂度为 $O(1)$。

2.2.3　本节试题精选

一、单项选择题

01. 下列叙述中，（　）是顺序存储结构的优点。

A. 存储密度大　　B. 插入运算方便

C. 删除运算方便　　D. 方便地运用于各种逻辑结构的存储表示

王道计算机教育 www.wangdaokaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

02. 下列关于顺序表的叙述中，正确的是（ ）。

A. 顺序表可以利用一维数组表示，因此顺序表与一维数组在逻辑结构上是相同的

B. 在顺序表中，逻辑上相邻的元素物理位置上不一定相邻

C. 顺序表和一维数组一样，都可以进行随机存取

D. 在顺序表中，每个元素的类型不必相同

03. 线性表的顺序存储结构是一种（ ）。

A. 随机存取的存储结构　　B. 顺序存取的存储结构

C. 索引存取的存储结构　　D. 散列存取的存储结构

04. 通常说顺序表具有随机存取的特性，指的是（ ）。

A. 查找值为 x 的元素的时间与顺序表中元素个数 n 无关

B. 查找值为 x 的元素的时间与顺序表中元素个数 n 有关

C. 查找序号为 i 的元素的时间与顺序表中元素个数 n 无关

D. 查找序号为 i 的元素的时间与顺序表中元素个数 n 有关

05. 一个顺序表所占用的存储空间大小与（ ）无关。

A. 表的长度　　B. 元素的存放顺序

C. 元素的类型　　D. 元素中各字段的类型

06. 若线性表最常用的操作是存取第 i 个元素及其前驱和后继元素的值，为了提高效率，应采用（ ）的存储方式。

A. 单链表　　B. 双链表　　C. 循环单链表　　D. 顺序表

07. 一个线性表最常用的操作是存取任意一个指定序号的元素并在最后进行插入、删除操作，则利用（ ）存储方式可以节省时间。

A. 顺序表　　B. 双链表

C. 带头结点的循环双链表　　D. 循环单链表

08. 在 n 个元素的线性表的数组表示中，时间复杂度为 $O(1)$ 的操作是（ ）。

I. 访问第 i（$1 \leqslant i \leqslant n$）个结点和求第 i（$2 \leqslant i \leqslant n$）个结点的直接前驱

II. 在最后一个结点后插入一个新的结点

III. 删除第 1 个结点

IV. 在第 i（$1 \leqslant i \leqslant n$）个结点后插入一个结点

A. I　　B. II、III　　C. I、II　　D. I、II、III

09. 设线性表有 n 个元素，严格说来，以下操作中，（ ）在顺序表上实现要比在链表上实现的效率高。

I. 输出第 i（$1 \leqslant i \leqslant n$）个元素值

II. 交换第 3 个元素与第 4 个元素的值

III. 顺序输出这 n 个元素的值

A. I　　B. I、III　　C. I、II　　D. II、III

10. 在一个长度为 n 的顺序表中删除第 i（$1 \leqslant i \leqslant n$）个元素时，需向前移动（ ）个元素。

A. n　　B. $i-1$　　C. $n-i$　　D. $n-i+1$

11. 对于顺序表，访问第 i 个位置的元素和在第 i 个位置插入一个元素的时间复杂度为（ ）。

A. $O(n)$，$O(n)$　　B. $O(n)$，$O(1)$　　C. $O(1)$，$O(n)$　　D. $O(1)$，$O(1)$

12. 对于顺序存储的线性表，其算法时间复杂度为 $O(1)$ 的运算应该是（ ）。

A. 将 n 个元素从小到大排序

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

B. 删除第 i（$1 \leqslant i \leqslant n$）个元素

C. 改变第 i（$1 \leqslant i \leqslant n$）个元素的值

D. 在第 i（$1 \leqslant i \leqslant n$）个元素后插入一个新元素

13. 若长度为 n 的非空线性表采用顺序存储结构，在表的第 i 个位置插入一个数据元素，则 i 的合法值应该是（　）。

A. $1 \leqslant i \leqslant n$　　B. $1 \leqslant i \leqslant n+1$　　C. $0 \leqslant i \leqslant n-1$　　D. $0 \leqslant i \leqslant n$

14. 顺序表的插入算法中，当 n 个空间已满时，可再申请增加分配 m 个空间，若申请失败，则说明系统没有（　）可分配的存储空间。

A. m 个　　B. m 个连续　　C. $n+m$ 个　　D. $n+m$ 个连续

15. 【2023 统考真题】在下列对顺序存储的有序表（长度为 n）实现给定操作的算法中，平均时间复杂度为 $O(1)$ 的是（　）。

A. 查找包含指定值元素的算法　　B. 插入包含指定值元素的算法

C. 删除第 i（$1 \leqslant i \leqslant n$）个元素的算法　　D. 获取第 i（$1 \leqslant i \leqslant n$）个元素的算法

二、综合应用题

01. 从顺序表中删除具有最小值的元素（假设唯一）并由函数返回被删元素的值。空出的位置由最后一个元素填补，若顺序表为空，则显示出错信息并退出运行。

02. 设计一个高效算法，将顺序表 L 的所有元素逆置，要求算法的空间复杂度为 $O(1)$。

03. 对长度为 n 的顺序表 L，编写一个时间复杂度为 $O(n)$、空间复杂度为 $O(1)$的算法，该算法删除顺序表中所有值为 x 的数据元素。

04. 从顺序表中删除其值在给定值 s 和 t 之间（包含 s 和 t，要求 $s<t$）的所有元素，若 s 或 t 不合理或顺序表为空，则显示出错信息并退出运行。

05. 从有序顺序表中删除所有其值重复的元素，使表中所有元素的值均不同。

06. 将两个有序顺序表合并为一个新的有序顺序表，并由函数返回结果顺序表。

07. 已知在一维数组 $A[m+n]$中依次存放两个线性表$(a_1, a_2, a_3, \cdots, a_m)$和$(b_1, b_2, b_3, \cdots, b_n)$。编写一个函数，将数组中两个顺序表的位置互换，即将$(b_1, b_2, b_3, \cdots, b_n)$放在$(a_1, a_2, a_3, \cdots, a_m)$的前面。

08. 线性表$(a_1, a_2, a_3, \cdots, a_n)$中的元素递增有序且按顺序存储于计算机内。要求设计一个算法，完成用最少时间在表中查找数值为 x 的元素，若找到，则将其与后继元素位置相交换，若找不到，则将其插入表中并使表中元素仍递增有序。

09. 给定三个序列 A、B、C，长度均为 n，且均为无重复元素的递增序列，请设计一个时间上尽可能高效的算法，逐行输出同时存在于这三个序列中的所有元素。例如，数组 A 为 $\{1, 2, 3\}$，数组 B 为$\{2, 3, 4\}$，数组 C 为$\{-1, 0, 2\}$，则输出 2。要求：

1）给出算法的基本设计思想。

2）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

3）说明你的算法的时间复杂度和空间复杂度。

10. 【2010 统考真题】设将 n（$n>1$）个整数存放到一维数组 R 中。设计一个在时间和空间两方面都尽可能高效的算法。将 R 中保存的序列循环左移 p（$0<p<n$）个位置，即将 R 中的数据由$(X_0, X_1, \cdots, X_{n-1})$变换为$(X_p, X_{p+1}, \cdots, X_{n-1}, X_0, X_1, \cdots, X_{p-1})$。要求：

1）给出算法的基本设计思想。

2）根据设计思想，采用 C 或 C++或 Java 语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。

11. 【2011 统考真题】一个长度为 L（$L \geqslant 1$）的升序序列 S，处在第$\lceil L/2 \rceil$个位置的数称为 S

配套课程公众号[考机研]
永久联系微信:4550060

的中位数。例如，若序列 $S_1=(11, 13, 15, 17, 19)$，则 S_1 的中位数是 15，两个序列的中位数是含它们所有元素的升序序列的中位数。例如，若 $S_2=(2, 4, 6, 8, 20)$，则 S_1 和 S_2 的中位数是 11。现在有两个等长升序序列 A 和 B，试设计一个在时间和空间两方面都尽可能高效的算法，找出两个序列 A 和 B 的中位数。要求：

1）给出算法的基本设计思想。

2）根据设计思想，采用 C 或 C++或 Java 语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。

12. 【2013 统考真题】已知一个整数序列 $A=(a_0, a_1, \cdots, a_{n-1})$，其中 $0\leqslant a_i<n$（$0\leqslant i<n$）。若存在 $a_{p1}=a_{p2}=\cdots=a_{pm}=x$ 且 $m>n/2$（$0\leqslant p_k<n$, $1\leqslant k\leqslant m$），则称 x 为 A 的主元素。例如 $A=(0, 5, 5, 3, 5, 7, 5, 5)$，则 5 为主元素；又如 $A=(0, 5, 5, 3, 5, 1, 5, 7)$，则 A 中没有主元素。假设 A 中的 n 个元素保存在一个一维数组中，请设计一个尽可能高效的算法，找出 A 的主元素。若存在主元素，则输出该元素；否则输出 -1。要求：

1）给出算法的基本设计思想。

2）根据设计思想，采用 C 或 C++或 Java 语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。

13. 【2018 统考真题】给定一个含 n（$n\geqslant 1$）个整数的数组，请设计一个在时间上尽可能高效的算法，找出数组中未出现的最小正整数。例如，数组{-5, 3, 2, 3}中未出现的最小正整数是 1；数组{1, 2, 3}中未出现的最小正整数是 4。要求：

1）给出算法的基本设计思想。

2）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。

14. 【2020 统考真题】定义三元组(a, b, c)（a, b, c 均为整数）的距离 $D=|a-b|+|b-c|+|c-a|$。给定 3 个非空整数集合 S_1、S_2 和 S_3，按升序分别存储在 3 个数组中。请设计一个尽可能高效的算法，计算并输出所有可能的三元组(a, b, c)（$a\in S_1$，$b\in S_2$，$c\in S_3$）中的最小距离。例如 $S_1=\{-1, 0, 9\}$，$S_2=\{-25, -10, 10, 11\}$，$S_3=\{2, 9, 17, 30, 41\}$，则最小距离为 2，相应的三元组为(9, 10, 9)。要求：

1）给出算法的基本设计思想。

2）根据设计思想，采用 C 语言或 C++语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。

2.2.4 答案与解析

一、单项选择题

01. A

顺序表不像链表那样要在结点中存放指针域，因此存储密度大，选项 A 正确。选项 B 和 C 是链表的优点。选项 D 是错误的，比如对于树形结构，顺序表显然不如链表表示起来方便。

02. C

顺序表是顺序存储的线性表，表中所有元素的类型必须相同，且必须连续存放。一维数组中的元素可以不连续存放；此外，栈、队列和树等逻辑结构也可利用一维数组表示，但它与顺序表不属于相同的逻辑结构。在顺序表中，逻辑上相邻的元素物理位置上也相邻。

03. A

本题易误选选项 B。注意，存取方式是指读/写方式。顺序表是一种支持随机存取的存储结

配套课程公众号[考机研]
永久联系微信:4550060

构，根据起始地址加上元素的序号，可以很方便地访问任意一个元素，这就是随机存取的概念。

04．C

随机存取是指在 $O(1)$的时间访问下标为 i 的元素，所需时间与顺序表中的元素个数 n 无关。

05．B

顺序表所占的存储空间 = 表长×sizeof（元素的类型），表长和元素的类型显然会影响存储空间的大小。若元素为结构体类型，则元素中各字段的类型也会影响存储空间的大小。

06．D

题干实际要求能最快存取第 $i-1$、i 和 $i+1$ 个元素值。选项 A、B、C 都只能从头结点依次顺序查找，时间复杂度为 $O(n)$；只有顺序表可以按序号随机存取，时间复杂度为 $O(1)$。

07．A

只有顺序表可以按序号随机存取，且在最后进行插入和删除操作时不需要移动任何元素。

08．C

对于 I，解析略；对于 II，在最后位置插入新结点不需要移动元素，时间复杂度为 $O(1)$；对于 III，被删结点后的结点需要依次前移，时间复杂度为 $O(n)$；对于 IV，需要后移 $n-i$ 个结点，时间复杂度为 $O(n)$。

09．C

对于 II，顺序表只需要 3 次交换操作；链表则需要分别找到两个结点前驱，第 4 个结点断链后再插入到第 2 个结点后，效率较低。对于 III，需依次顺序访问每个元素，时间复杂度相同。

10．C

需要将元素 $a_{i+1}\sim a_n$ 依次前移一位，共移动 $n-(i+1)+1=n-i$ 个元素。

11．C

在第 i 个位置插入一个元素，需要移动 $n-i+1$ 个元素，时间复杂度为 $O(n)$。

12．C

对 n 个元素进行排序的时间复杂度最小也要 $O(n)$（初始有序时），通常为 $O(n\log_2 n)$或 $O(n^2)$，通过第 8 章学习后会更理解。选项 B 和 D 显然错误。顺序表支持按序号的随机存取方式。

13．B

线性表元素的序号是从 1 开始，而在第 $n+1$ 个位置插入相当于在表尾追加。

14．D

顺序存储需要连续的存储空间，在申请时需申请 $n+m$ 个连续的存储空间，然后将线性表原来的 n 个元素复制到新申请的 $n+m$ 个连续的存储空间的前 n 个单元。

15．D

对于顺序存储的有序表，查找指定值元素可以采用顺序查找法或折半查找法，平均时间复杂度最少为 $O(\log_2 n)$。插入指定值元素需要先找到插入位置，然后将该位置及之后的元素依次后移一个位置，最后将指定值元素插入到该位置，平均时间复杂度为 $O(n)$。删除第 i 个元素需要将该元素之后的全部元素依次前移一个位置，平均时间复杂度为 $O(n)$。获取第 i 个元素只需直接根据下标读取对应的数组元素即可，时间复杂度为 $O(1)$。

二、综合应用题

01．【解答】

算法思想：搜索整个顺序表，查找最小值元素并记住其位置，搜索结束后用最后一个元素填补空出的原最小值元素的位置。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

本题代码如下：

```
bool Del_Min(SqList &L,ElemType &value){
//删除顺序表 L 中最小值元素结点，并通过引用型参数 value 返回其值
//若删除成功，则返回 true；否则返回 false
    if(L.length==0)
        return  false;                      //表空，中止操作返回
    value=L.data[0];
    int pos=0;                              //假定 0 号元素的值最小
    for(int i=1;i<L.length;i++)             //循环，寻找具有最小值的元素
        if(L.data[i]<value){                //让 value 记忆当前具有最小值的元素
            value=L.data[i];
            pos=i;
        }
    L.data[pos]=L.data[L.length-1];         //空出的位置由最后一个元素填补
    L.length--;
    return  true;                           //此时，value 为最小值
}
```

注　意

本题也可用函数返回值返回，两者的区别是：函数返回值只能返回一个值，而参数返回（引用传参）可以返回多个值。

02.【解答】

算法思想：扫描顺序表 L 的前半部分元素，对于元素 L.data[i]（0<=i<L.length/2），将其与后半部分的对应元素 L.data[L.length-i-1]进行交换。

本题代码如下：

```
void Reverse(SqList &L){
    ElemType temp;                  //辅助变量
    for(int i=0;i<L.length/2;i++){
        temp=L.data[i];             //交换 L.data[i]与 L.data[L.length-i-1]
        L.data[i]=L.data[L.length-i-1];
        L.data[L.length-i-1]=temp;
    }
}
```

03.【解答】

解法 1：用 k 记录顺序表 L 中不等于 x 的元素个数（需要保存的元素个数），扫描时将不等于 x 的元素移动到下标 k 的位置，并更新 k 值。扫描结束后修改 L 的长度。

本题代码如下：

```
void del_x_1(SqList &L,ElemType x){
//本算法实现删除顺序表 L 中所有值为 x 的数据元素
    int k=0,i;                          //记录值不等于 x 的元素个数
    for(i=0;i<L.length;i++)
        if(L.data[i]!=x){
            L.data[k]=L.data[i];
            k++;                        //不等于 x 的元素增 1
        }
    L.length=k;                         //顺序表 L 的长度等于 k
}
```

解法 2：用 k 记录顺序表 L 中等于 x 的元素个数，一边扫描 L，一边统计 k，并将不等于 x 的元素前移 k 个位置。扫描结束后修改 L 的长度。

本题代码如下：

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

```
void del_x_2(SqList &L,ElemType x){
    int k=0,i=0;                          //k 记录值等于 x 的元素个数
    while(i<L.length){
        if(L.data[i]==x)
            k++;
        else
            L.data[i-k]=L.data[i];  //当前元素前移 k 个位置
        i++;
        }
    L.length=L.length-k;                  //顺序表 L 的长度递减
}
```

此外，本题还可以考虑设头、尾两个指针（$i=1, j=n$），从两端向中间移动，在遇到最左端值 x 的元素时，直接将最右端值非 x 的元素左移至值为 x 的数据元素位置，直到两指针相遇。但这种方法会改变原表中元素的相对位置。

04.【解答】

算法思想：从前向后扫描顺序表 L，用 k 记录值在 s 和 t 之间的元素个数（初始时 $k=0$）。对于当前扫描的元素，若其值不在 s 和 t 之间，则前移 k 个位置；否则执行 k++。每个不在 s 和 t 之间的元素仅移动一次，因此算法效率高。

本题代码如下：

```
bool Del_s_t(SqList &L,ElemType s,ElemType t){
//删除顺序表 L 中值在给定值 s 和 t（要求 s<t）之间的所有元素
    int i,k=0;
    if(L.length==0||s>=t)
        return false;                     //线性表为空或 s、t 不合法，返回
    for(i=0;i<L.length;i++){
        if(L.data[i]>=s&&L.data[i]<=t)
            k++;
        else
            L.data[i-k]=L.data[i];  //当前元素前移 k 个位置
    } //for
    L.length-=k;                          //长度减小
    return  true;
}
```

注　意

本题也可从后向前扫描顺序表，每遇到一个值在 s 和 t 之间的元素，就删除该元素，其后的所有元素全部前移。但移动次数远大于前者，效率不够高。

05.【解答】

算法思想：注意是有序顺序表，值相同的元素一定在连续的位置上，用类似于直接插入排序的思想，初始时将第一个元素视为非重复的有序表。之后依次判断后面的元素是否与前面非重复有序表的最后一个元素相同，若相同，则继续向后判断，若不同，则插入前面的非重复有序表的最后，直至判断到表尾为止。

本题代码如下：

```
bool  Delete_Same(SeqList& L){
   if(L.length==0)
      return  false;
   int i,j;                               //i 存储第一个不相同的元素，j 为工作指针
   for(i=0,j=1;j<L.length;j++)
      if(L.data[i]!=L.data[j])            //查找下一个与上一个元素值不同的元素
         L.data[++i]=L.data[j];           //找到后，将元素前移
```

配套课程公众号[考机研]
永久联系微信:4550060

```
        L.length=i+1;
        return true;
    }
```

对于本题的算法，请读者用序列 1, 2, 2, 2, 2, 3, 3, 3, 4, 4, 5 来手动模拟算法的执行过程，在模拟过程中要标注 i 和 j 所指示的元素。

思考：若将本题中的有序表改为无序表，你能想到时间复杂度为 $O(n)$的方法吗？

（提示：使用散列表。）

06.【解答】

算法思想：首先，按顺序不断取下两个顺序表表头较小的结点存入新的顺序表中。然后，看哪个表还有剩余，将剩下的部分加到新的顺序表后面。

本题代码如下：

```
bool  Merge(SeqList A,SeqList B,SeqList &C){
    //将有序顺序表 A 与 B 合并为一个新的有序顺序表 C
    if(A.length+B.length>C.maxSize)      //大于顺序表的最大长度
        return  false;
    int i=0,j=0,k=0;
    while(i<A.length&&j<B.length){       //循环，两两比较，小者存入结果表
        if(A.data[i]<=B.data[j])
            C.data[k++]=A.data[i++];
        else
            C.data[k++]=B.data[j++];
    }
    while(i<A.length)                    //还剩一个没有比较完的顺序表
        C.data[k++]=A.data[i++];
    while(j<B.length)
        C.data[k++]=B.data[j++];
    C.length=k;
    return  true;
}
```

王道计算机教育 www.cskaoyan.com

注 意

本算法的方法非常典型，需牢固掌握。

07.【解答】

算法思想：首先将数组 $A[m+n]$中的全部元素$(a_1, a_2, a_3, \cdots, a_m, b_1, b_2, b_3, \cdots, b_n)$原地逆置为$(b_n, b_{n-1}, b_{n-2}, \cdots, b_1, a_m, a_{m-1}, a_{m-2}, \cdots, a_1)$，然后对前 n 个元素和后 m 个元素分别使用逆置算法，即可得到$(b_1, b_2, b_3, \cdots, b_n, a_1, a_2, a_3, \cdots, a_m)$，从而实现顺序表的位置互换。

本题代码如下：

```
typedef int DataType;
void Reverse(DataType A[],int left,int right,int arraySize){
//逆转(aleft,aleft+1,aleft+2,…,aright)为(aright,aright-1,…,aleft)
    if(left>=right||right>=arraySize)
        return;
    int mid=(left+right)/2;
    for(int i=0;i<=mid-left;i++){
        DataType temp=A[left+i];
        A[left+i]=A[right-i];
        A[right-i]=temp;
    }
}
void Exchange(DataType A[],int m,int n,int arraySize){
/*数组 A[m+n]中，从 0 到 m-1 存放顺序表(a1,a2,a3,…,am)，从 m 到 m+n-1 存放顺序表
```

配套课程公众号[考机研]
永久联系微信:4550060

```
(b1,b2,b3,…,bn)，算法将这两个表的位置互换*/
    Reverse(A,0,m+n-1,arraySize);
    Reverse(A,0,n-1,arraySize);
    Reverse(A,n,m+n-1,arraySize);
}
```

08.【解答】

算法思想：顺序存储的线性表递增有序，可以顺序查找，也可以折半查找。题目要求“用最少的时间在表中查找数值为 x 的元素”，这里应使用折半查找法。

本题代码如下：

```
void SearchExchangeInsert(ElemType A[],ElemType x){
    int low=0,high=n-1,mid;          //low 和 high 指向顺序表下界和上界的下标
    while(low<=high){
        mid=(low+high)/2;            //找中间位置
        if(A[mid]==x) break;         //找到 x，退出 while 循环
        else if(A[mid]<x) low=mid+1;//到中点 mid 的右半部去查
        else high=mid-1;             //到中点 mid 的左半部去查
    }   //下面两个 if 语句只会执行一个
    if(A[mid]==x&&mid!=n-1){         //若最后一个元素与 x 相等，则不存在与其后
                                       继交换的操作
        t=A[mid]; A[mid]=A[mid+1]; A[mid+1]=t;
    }
    if(low>high){                                  //查找失败，插入数据元素 x
        for(i=n-1;i>high;i--) A[i+1]=A[i];         //后移元素
        A[i+1]=x;                                  //插入 x
    }                                              //结束插入
}
```

本题的算法也可写成三个函数：查找函数、交换后继函数与插入函数。写成三个函数的优点是逻辑清晰、易读。

09.【解析】

1）算法的基本设计思想。

使用三个下标变量从小到大遍历数组。当三个下标变量指向的元素相等时，输出并向前推进指针，否则仅移动小于最大元素的下标变量，直到某个下标变量移出数组范围，即可停止。

2）算法的实现。

```
void samekey(int A[],int B[],int C[],int n){
    int i=0,j=0,k=0;                 //定义三个工作指针
    while(i<n&&j<n&&k<n){            //相同则输出，并集体后移
        if(A[i]==B[j]&&B[j]==C[k]){
            printf("%d\n",A[i]);
            i++;j++;k++;
        }else{
            int maxNum=max(A[i],max(B[j],C[k]));
            if(A[i]<maxNum)i++;
            if(B[j]<maxNum)j++;
            if(C[k]<maxNum)k++;
        }
    }
}
```

3）每个指针移动的次数不超过 n 次，且每次循环至少有一个指针后移，所以时间复杂度为 $O(n)$，算法只用到了常数个变量，空间复杂度为 $O(1)$。

10.【解答】

1）算法的基本设计思想：

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

可将问题视为把数组 ab 转换成数组 ba（a 代表数组的前 p 个元素，b 代表数组中余下的 $n-p$ 个元素），先将 a 逆置得到 $a^{-1}b$，再将 b 逆置得到 $a^{-1}b^{-1}$，最后将整个 $a^{-1}b^{-1}$ 逆置得到$(a^{-1}b^{-1})^{-1}=ba$。设 Reverse 函数执行将数组逆置的操作，对 abcdefgh 向左循环移动 3（$p=3$）个位置的过程如下：

Reverse(0,p-1)得到 cbadefgh；

Reverse(p,n-1)得到 cbahgfed；

Reverse(0,n-1)得到 defghabc。

注：在 Reverse 中，两个参数分别表示数组中待转换元素的始末位置。

2）使用 C 语言描述算法如下：

```
void Reverse(int R[],int from,int to) {
    int i,temp;
    for(i=0;i<(to-from+1)/2;i++)
    {temp=R[from+i];R[from+i]=R[to-i];R[to-i]=temp;}
}
void Converse(int R[],int n,int p){
    Reverse(R,0,p-1);
    Reverse(R,p,n-1);
    Reverse(R,0,n-1);
}
```

3）上述算法中三个 Reverse 函数的时间复杂度分别为 $O(p/2)$、$O((n-p)/2)$和 $O(n/2)$，故所设计的算法的时间复杂度为 $O(n)$，空间复杂度为 $O(1)$。

【另解】借助辅助数组来实现。算法思想：创建大小为 p 的辅助数组 S，将 R 中前 p 个整数依次暂存在 S 中，同时将 R 中后 $n-p$ 个整数左移，然后将 S 中暂存的 p 个数依次放回到 R 中的后续单元。时间复杂度为 $O(n)$，空间复杂度为 $O(p)$。

11.【解答】

1）算法的基本设计思想如下。

分别求两个升序序列 A、B 的中位数，设为 a 和 b，求序列 A、B 的中位数过程如下：

① 若 $a=b$，则 a 或 b 为所求中位数，算法结束。

② 若 $a<b$，则舍弃序列 A 中较小的一半，同时舍弃序列 B 中较大的一半，要求两次舍弃的长度相等。

③ 若 $a>b$，则舍弃序列 A 中较大的一半，同时舍弃序列 B 中较小的一半，要求两次舍弃的长度相等。

在保留的两个升序序列中，重复过程①、②、③，直到两个序列中均只含一个元素时为止，较小者为所求的中位数。

2）本题代码如下：

```
int M_Search(int A[],int B[],int n){
    int s1,d1,m1,s2,d2,m2;
    s1=0;d1=n-1;
    s2=0;d2=n-1;
    while(s1!=d1||s2!=d2){
        m1=(s1+d1)/2;
        m2=(s2+d2)/2;
        if(A[m1]==B[m2])
            return A[m1];          //满足条件①
        if(A[m1]<B[m2]){           //满足条件②
            if((s1+d1)%2==0){      //若元素个数为奇数
                s1=m1;             //舍弃 A 中间点以前的部分，且保留中间点
                d2=m2;             //舍弃 B 中间点以后的部分，且保留中间点
            }
```

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

```
                else{                   //元素个数为偶数
                    s1=m1+1;            //舍弃 A 的前半部分
                    d2=m2;              //舍弃 B 的后半部分
                }
            }
            else{                       //满足条件③
                if((s1+d1)%2==0){       //若元素个数为奇数
                    d1=m1;              //舍弃 A 中间点以后的部分，且保留中间点
                    s2=m2;              //舍弃 B 中间点以前的部分，且保留中间点
                }
                else{                   //元素个数为偶数
                    d1=m1;              //舍弃 A 的后半部分
                    s2=m2+1;            //舍弃 B 的前半部分
                }
            }
        }
        return A[s1]<B[s2]? A[s1]:B[s2];
    }
```

3）算法的时间复杂度为 $O(\log_2 n)$，空间复杂度为 $O(1)$。

【另解】对两个长度为 n 的升序序列 A 和 B 中的元素按从小到大的顺序依次访问，这里访问的含义只是比较序列中两个元素的大小，并不实现两个序列的合并，因此空间复杂度为 $O(1)$。按照上述规则访问第 n 个元素时，这个元素为两个序列 A 和 B 的中位数。

12.【解答】

1）算法的基本设计思想：算法的策略是从前向后扫描数组元素，标记出一个可能成为主元素的元素 Num。然后重新计数，确认 Num 是否是主元素。

算法可分为以下两步：

① 选取候选的主元素。依次扫描所给数组中的每个整数，将第一个遇到的整数 Num 保存到 c 中，记录 Num 的出现次数为 1；若遇到的下一个整数仍等于 Num，则计数加 1，否则计数减 1；当计数减到 0 时，将遇到的下一个整数保存到 c 中，计数重新记为 1，开始新一轮计数，即从当前位置开始重复上述过程，直到扫描完全部数组元素。

② 判断 c 中元素是否是真正的主元素。再次扫描该数组，统计 c 中元素出现的次数，若大于 $n/2$，则为主元素；否则，序列中不存在主元素。

2）算法实现如下：

```
int Majority(int A[],int n){
    int i,c,count=1;                //c 用来保存候选主元素，count 用来计数
    c=A[0];                         //设置 A[0]为候选主元素
    for(i=1;i<n;i++)                //查找候选主元素
        if(A[i]==c)
            count++;                //对 A 中的候选主元素计数
        else
            if(count>0)             //处理不是候选主元素的情况
                count--;
            else{                   //更换候选主元素，重新计数
                c=A[i];
                count=1;
            }
    if(count>0)
        for(i=count=0;i<n;i++)      //统计候选主元素的实际出现次数
            if(A[i]==c)
                count++;
    if(count>n/2) return c;         //确认候选主元素
```

配套课程公众号[考机研]
永久联系微信:4550060

```
        else return -1;                    //不存在主元素
    }
```

3）实现的程序的时间复杂度为 $O(n)$，空间复杂度为 $O(1)$。

说　明

本题若采用先排好序再统计的方法［时间复杂度为 $O(n\log_2 n)$］，则只要解答正确，最高可拿 11 分。即便是写出 $O(n^2)$的算法，最高也能拿 10 分，因此对于统考算法题，花费大量时间去思考最优解法是得不偿失的。本算法的方法非常典型，需牢固掌握。

13.【解答】

1）算法的基本设计思想：

要求在时间上尽可能高效，因此采用空间换时间的办法。分配一个用于标记的数组 B[n]，用来记录 A 中是否出现了 1~n 中的正整数，B[0]对应正整数 1，B[n-1]对应正整数 n，初始化 B 中全部为 0。A 中含有 n 个整数，因此可能返回的值是 1~n+1，当 A 中 n 个数恰好为 1~n 时返回 n+1。当数组 A 中出现了小于或等于 0 或大于 n 的值时，会导致 1~n 中出现空余位置，返回结果必然在 1~n 中，因此对于 A 中出现了小于或等于 0 或大于 n 的值，可以不采取任何操作。

经过以上分析可以得出算法流程：从 A[0]开始遍历 A，若 0<A[i]<=n，则令 B[A[i]-1]=1；否则不做操作。对 A 遍历结束后，开始遍历数组 B，若能查找到第一个满足 B[i]==0 的下标 i，返回 i+1 即为结果，此时说明 A 中未出现的最小正整数在 1 和 n 之间。若 B[i]全部不为 0，返回 i+1（跳出循环时 i=n，i+1 等于 n+1），此时说明 A 中未出现的最小正整数是 n+1。

2）算法实现：

```
int findMissMin(int A[],int n)
{
    int i,*B;                          //标记数组
    B=(int *)malloc(sizeof(int)*n);//分配空间
    memset(B,0,sizeof(int)*n);         //赋初值为 0
    for(i=0;i<n;i++)
        if(A[i]>0&&A[i]<=n)            //若 A[i]的值介于 1~n，则标记数组 B
            B[A[i]-1]=1;
    for(i=0;i<n;i++)                   //扫描数组 B，找到目标值
        if (B[i]==0) break;
    return i+1;                        //返回结果
}
```

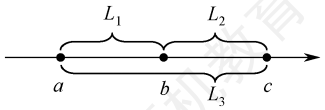
3）时间复杂度：遍历 A 一次，遍历 B 一次，两次循环内操作步骤为 $O(1)$量级，因此时间复杂度为 $O(n)$。空间复杂度：额外分配了 B[n]，空间复杂度为 $O(n)$。

14.【解答】

分析。由 $D=|a-b|+|b-c|+|c-a|\geqslant 0$ 有如下结论。

① 当 $a=b=c$ 时，距离最小。

② 其余情况。不失一般性，假设 $a\leqslant b\leqslant c$，观察下面的数轴：



$$L_1=|a-b|,\quad L_2=|b-c|,\quad L_3=|c-a|$$

$$D=|a-b|+|b-c|+|c-a|\geqslant 0=L_1+L_2+L_3=2L_3$$

由 D 的表达式可知，事实上决定 D 大小的关键是 a 和 c 之间的距离，于是问题就可以简化

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

为每次固定 c 找一个 a，使得 $L_3=|c-a|$ 最小。

1）算法的基本设计思想：

① 使用 D_{min} 记录所有已处理的三元组的最小距离，初值为一个足够大的整数。

② 集合 S_1、S_2 和 S_3 分别保存在数组 A、B、C 中。数组的下标变量 $i=j=k=0$，当 $i<|S_1|$、$j<|S_2|$且 $k<|S_3|$时（$|S|$表示集合 S 中的元素个数），循环执行下面的 a）～c）。

a）计算$(A[i], B[j], C[k])$的距离 D；（计算 D）

b）若 $D<D_{min}$，则 $D_{min}=D$；（更新 D）

c）将 $A[i]$、$B[j]$、$C[k]$中的最小值的下标+1；　（对照分析：最小值为 a，最大值为 c，这里 c 不变而更新 a，试图寻找更小的距离 D）

③ 输出 D_{min}，结束。

2）算法实现：

```
#define INT_MAX 0x7fffffff
int abs_(int a){//计算绝对值
    if(a<0) return -a;
    else return a;
}
bool xls_min(int a,int b,int c){//a 是否是三个数中的最小值
    if(a<=b&&a<=c) return true;
    return false;
}
int findMinofTrip(int A[],int n,int B[],int m,int C[],int p){
    //D_min 用于记录三元组的最小距离，初值赋为 INT_MAX
    int i=0,j=0,k=0,D_min=INT_MAX,D;
    while(i<n&&j<m&&k<p&&D_min>0){
        D=abs_(A[i]-B[j])+abs_(B[j]-C[k])+abs_(C[k]-A[i]);//计算 D
        if(D<D_min) D_min=D;                    //更新 D
        if(xls_min(A[i],B[j],C[k]))  i++;  //更新 a
        else if(xls_min(B[j],C[k],A[i]))  j++;
        else k++;
    }
    return D_min;
}
```

3）设 $n=(|S_1|+|S_2|+|S_3|)$，时间复杂度为 $O(n)$，空间复杂度为 $O(1)$。

2.3　线性表的链式表示

顺序表的存储位置可以用一个简单直观的公式表示，它可以随机存取表中任一元素，但插入和删除操作需要移动大量元素。链式存储线性表时，不需要使用地址连续的存储单元，即不要求逻辑上相邻的元素在物理位置上也相邻，它通过“链”建立元素之间的逻辑关系，因此插入和删除操作不需要移动元素，而只需修改指针，但也会失去顺序表可随机存取的优点。

2.3.1　单链表的定义

命题追踪 ▶▶ 单链表的应用（2009、2012、2013、2015、2016、2019）

线性表的链式存储也称单链表，它是指通过一组任意的存储单元来存储线性表中的数据元素。为了建立数据元素之间的线性关系，对每个链表结点，除存放元素自身的信息外，还需要存放一个指向其后继的指针。单链表结点结构如图 2.3 所示，其中 data 为数据域，存放数据元素；

配套课程公众号[考机研]
永久联系微信:4550060

next为指针域，存放其后继结点的地址。

图2.3 单链表结点结构

单链表中结点类型的描述如下：

```
typedef struct LNode{          //定义单链表结点类型
   ElemType data;              //数据域
   struct LNode *next;         //指针域
}LNode, *LinkList;
```

利用单链表可以解决顺序表需要大量连续存储单元的缺点，但附加的指针域，也存在浪费存储空间的缺点。单链表的元素离散地分布在存储空间中，因此是非随机存取的存储结构，即不能直接找到表中某个特定结点。查找特定结点时，需要从表头开始遍历，依次查找。

通常用头指针L（或head等）来标识一个单链表，指出链表的起始地址，头指针为NULL时表示一个空表。此外，为了操作上的方便，在单链表第一个数据结点之前附加一个结点，称为头结点。头结点的数据域可以不设任何信息，但也可以记录表长等信息。单链表带头结点时，头指针L指向头结点，如图2.4(a)所示。单链表不带头结点时，头指针L指向第一个数据结点，如图2.4(b)所示。表尾结点的指针域为NULL（用“^”表示）。

(a) 带表头结点的单链表　　(b) 不带表头结点的单链表

图2.4 带头结点和不带头结点的单链表

头结点和头指针的关系：不管带不带头结点，头指针都始终指向链表的第一个结点，而头结点是带头结点的链表中的第一个结点，结点内通常不存储信息。

引入头结点后，可以带来两个优点：

① 第一个数据结点的位置被存放在头结点的指针域中，因此在链表的第一个位置上的操作和在表的其他位置上的操作一致，无须进行特殊处理。

② 无论链表是否为空，其头指针都是指向头结点的非空指针（空表中头结点的指针域为空），因此空表和非空表的处理也就得到了统一。

2.3.2 单链表上基本操作的实现

带头结点单链表的操作代码书写较为方便，如无特殊说明，本节均默认链表带头结点。

1. 单链表的初始化

带头结点和不带头结点的单链表的初始化操作是不同的。带头结点的单链表初始化时，需要创建一个头结点，并让头指针指向头结点，头结点的next域初始化为NULL。

```
bool InitList(LinkList &L){                //带头结点的单链表的初始化
   L=(LNode*)malloc(sizeof(LNode));        //创建头结点①
   L->next=NULL;                           //头结点之后暂时还没有元素结点
   return true;
}
```

不带头结点的单链表初始化时，只需将头指针L初始化为NULL。

```
bool InitList(LinkList &L){                //不带头结点的单链表的初始化
```

① 执行s=(LNode*)malloc(sizeof(LNode))的作用是由系统生成一个LNode型的结点，并将该结点的起始位置赋给指针变量s。

配套课程公众号[考机研]
永久联系微信:4550060

```
        L=NULL;
        return true;
    }
```

注　意

设 p 为指向链表结点的结构体指针，则*p 表示结点本身，因此可用 p->data 或(*p).data 访问*p 这个结点的数据域，二者完全等价。成员运算符（.）左边是一个普通的结构体变量，而指向运算符（->）左边是一个结构体指针。通过(*p).next 可以得到指向下一个结点的指针，因此(*(*p).next).data 就是下一个结点中存放的数据，或者直接用 p->next->data。

2．求表长操作

求表长操作是计算单链表中数据结点的个数，需要从第一个结点开始依次访问表中每个结点，为此需设置一个计数变量，每访问一个结点，其值加 1，直到访问到空结点为止。

```
int Length(LinkList L){
    int len=0;                      //计数变量，初始为 0
    LNode *p=L;
    while(p->next!=NULL){
        p=p->next;
        len++;                      //每访问一个结点，计数加 1
    }
    return len;
}
```

求表长操作的时间复杂度为 $O(n)$。另需注意的是，因为单链表的长度是不包括头结点的，因此不带头结点和带头结点的单链表在求表长操作上会略有不同。

3．按序号查找结点

从单链表的第一个结点开始，沿着 next 域从前往后依次搜索，直到找到第 i 个结点为止，则返回该结点的指针；若 i 大于单链表的表长，则返回 NULL。

```
LNode *GetElem(LinkList L,int i){
    LNode *p=L;                     //指针 p 指向当前扫描到的结点
    int j=0;                        //记录当前结点的位序，头结点是第 0 个结点
    while(p!=NULL&&j<i){            //循环找到第 i 个结点
        p=p->next;
        j++;
    }
    return  p;                      //返回第 i 个结点的指针或 NULL
}
```

按序号查找操作的时间复杂度为 $O(n)$。

4．按值查找表结点

从单链表的第一个结点开始，从前往后依次比较表中各结点的数据域，若某结点的 data 域等于给定值 e，则返回该结点的指针；若整个单链表中没有这样的结点，则返回 NULL。

```
LNode *LocateElem(LinkList L,ElemType e){
    LNode *p=L->next;
    while(p!=NULL&&p->data!=e)  //从第一个结点开始查找数据域为 e 的结点
        p=p->next;
    return  p;                      //找到后返回该结点指针，否则返回 NULL
}
```

按值查找操作的时间复杂度为 $O(n)$。

5．插入结点操作

插入结点操作将值为 x 的新结点插入到单链表的第 i 个位置。先检查插入位置的合法性，然

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

后找到待插入位置的前驱，即第 i-1 个结点，再在其后插入。其操作过程如图 2.5 所示。

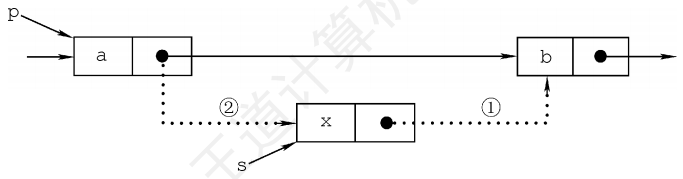


图 2.5 单链表的插入操作

命题追踪 ▶▶ 单链表插入操作的过程（2016、2024）

首先查找第 i-1 个结点，假设第 i-1 个结点为*p，然后令新结点*s 的指针域指向*p 的后继，再令结点*p 的指针域指向新插入的结点*s。

```
bool ListInsert(LinkList &L,int i,ElemType e){
    LNode *p=L;                          //指针 p 指向当前扫描到的结点
    int j=0;                             //记录当前结点的位序，头结点是第 0 个结点
    while(p!=NULL&&j<i-1){               //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL)                          //i 值不合法
        return  false;
    LNode *s=(LNode*)malloc(sizeof(LNode));
    s->data=e;
    s->next=p->next;                     //图 2.5 中操作步骤①
    p->next=s;                           //图 2.5 中操作步骤②
    return  true;
}
```

插入时，①和②的顺序不能颠倒，否则，先执行 p->next=s 后，指向其原后继的指针就不存在了，再执行 s->next=p->next 时，相当于执行了 s->next=s，显然有误。本算法主要的时间开销在于查找第 i-1 个元素，时间复杂度为 $O(n)$。若在指定结点后插入新结点，则时间复杂度仅为 $O(1)$。需注意的是，当链表不带头结点时，需要判断插入位置 i 是否为 1，若是，则要做特殊处理，将头指针 L 指向新的首结点。当链表带头结点时，插入位置 i 为 1 时不用做特殊处理。

扩展：对某一结点进行前插操作。

前插操作是指在某结点的前面插入一个新结点，后插操作的定义刚好与之相反。在单链表插入算法中，通常都采用后插操作。以上面的算法为例，先找到第 i-1 个结点，即插入结点的前驱，再对其执行后插操作。由此可知，对结点的前插操作均可转化为后插操作，前提是从单链表的头结点开始顺序查找到其前驱结点，时间复杂度为 $O(n)$。

此外，可采用另一种方式将其转化为后插操作来实现，设待插入结点为*s，将*s 插入到*p 的前面。我们仍然将*s 插入到*p 的后面，然后将 p->data 与 s->data 交换，这样做既满足逻辑关系，又能使得时间复杂度为 $O(1)$。该方法的主要代码片段如下：

```
s->next=p->next;                         //修改指针域，不能颠倒
p->next=s;
temp=p->data;                            //交换数据域部分
p->data=s->data;
s->data=temp;
```

6．删除结点操作

删除结点操作是将单链表的第 i 个结点删除。先检查删除位置的合法性，然后查找表中第 i－1 个结点，即被删结点的前驱，再删除第 i 个结点。其操作过程如图 2.6 所示。

配套课程公众号[考机研]
永久联系微信:4550060

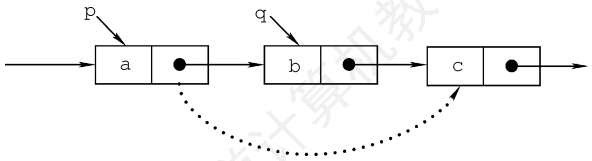


图 2.6　单链表结点的删除

假设结点*p 为找到的被删结点的前驱，为实现这一操作后的逻辑关系的变化，仅需修改*p 的指针域，将*p 的指针域 next 指向*q 的下一结点，然后释放*q 的存储空间。

```
bool ListDelete(LinkList &L,int i,ElemType &e){
    LNode *p=L;                    //指针 p 指向当前扫描到的结点
    int j=0;                       //记录当前结点的位序，头结点是第 0 个结点
    while(p->next!=NULL&&j<i-1){          //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p->next==NULL||j>i-1)       //i 值不合法
        return  false;
    LNode *q=p->next;              //令 q 指向被删除结点
    e=q->data;                     //用 e 返回元素的值
    p->next=q->next;               //将*q 结点从链中“断开”
    free(q);                       //释放结点的存储空间①
    return  true;
}
```

同插入算法一样，该算法的主要时间也耗费在查找操作上，时间复杂度为 $O(n)$。当链表不带头结点时，需要判断被删结点是否为首结点，若是，则要做特殊处理，将头指针 L 指向新的首结点。当链表带头结点时，删除首结点和删除其他结点的操作是相同的。

扩展：删除结点*p。

要删除某个给定结点*p，通常的做法是先从链表的头结点开始顺序找到其前驱，然后执行删除操作。其实，删除结点*p 的操作可用删除*p 的后继来实现，实质就是将其后继的值赋予其自身，然后再删除后继，也能使得时间复杂度为 $O(1)$。该方法的主要代码片段如下：

```
q=p->next;                    //令 q 指向*p 的后继结点
p->data=p->next->data;        //用后继结点的数据域覆盖
p->next=q->next;              //将*q 结点从链中“断开”
free(q);                      //释放后继结点的存储空间
```

7．采用头插法建立单链表

该方法从一个空表开始，生成新结点，并将读取到的数据存放到新结点的数据域中，然后将新结点插入到当前链表的表头，即头结点之后，如图 2.7 所示。算法实现如下：

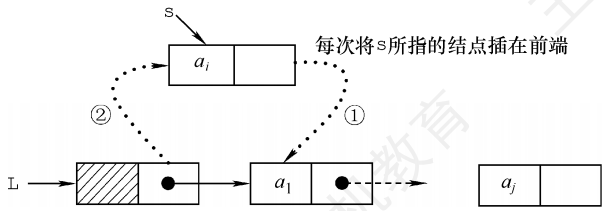


图 2.7　采用头插法建立单链表

① 执行 free(q)的作用是由系统回收一个 LNode 型结点，回收后的空间可供再次生成结点时用。

配套课程公众号[考机研]
永久联系微信:4550060

```
LinkList List_HeadInsert(LinkList &L){ //逆向建立单链表
    LNode *s; int x;                   //设元素类型为整型
    L=(LNode*)malloc(sizeof(LNode));   //创建头结点
    L->next=NULL;                      //初始为空链表
    scanf("%d",&x);                    //输入结点的值
    while(x!=9999){                    //输入 9999 表示结束
        s=(LNode*)malloc(sizeof(LNode));//创建新结点
        s->data=x;
        s->next=L->next;
        L->next=s;                     //将新结点插入表中，L 为头指针
        scanf("%d",&x);
    }
    return L;
}
```

采用头插法建立单链表时，读入数据的顺序与生成的链表中元素的顺序是相反的，可用来实现链表的逆置。每个结点插入的时间为 $O(1)$，设单链表长为 n，则总时间复杂度为 $O(n)$。

思 考

若单链表不带头结点，则上述代码中哪些地方需要修改？[①]

8．采用尾插法建立单链表

头插法建立单链表的算法虽然简单，但生成的链表中结点的次序和输入数据的顺序不一致。若希望两者次序一致，则可采用尾插法。该方法将新结点插入到当前链表的表尾，为此必须增加一个尾指针 r，使其始终指向当前链表的尾结点，如图 2.8 所示。算法实现如下：

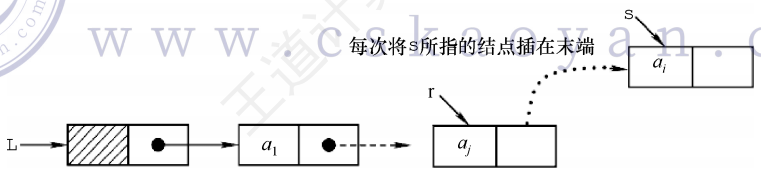


图 2.8　采用尾插法建立单链表

```
LinkList List_TailInsert(LinkList &L){ //正向建立单链表
    int x;                             //设元素类型为整型
    L=(LNode*)malloc(sizeof(LNode));   //创建头结点
    LNode *s,*r=L;                     //r 为表尾指针
    scanf("%d",&x);                    //输入结点的值
    while(x!=9999){                    //输入 9999 表示结束
        s=(LNode *)malloc(sizeof(LNode));
        s->data=x;
        r->next=s;
        r=s;                           //r 指向新的表尾结点
        scanf("%d",&x);
    }
    r->next=NULL;                      //尾结点指针置空
    return L;
}
```

因为附设了一个指向表尾结点的指针，所以时间复杂度和头插法的相同。

① 主要修改之处：因为在头部插入新结点，每次插入新结点后，都需要将它的地址赋值给头指针 L。

配套课程公众号[考机研]
永久联系微信:4550060

注　意

单链表是整个链表的基础，读者一定要熟练掌握单链表的基本操作算法。在设计算法时，建议先通过画图的方法理清算法的思路，然后进行算法的编写。

2.3.3　双链表

单链表结点中只有一个指向其后继的指针，使得单链表只能从前往后依次遍历。要访问某个结点的前驱（插入、删除操作时），只能从头开始遍历，访问前驱的时间复杂度为 $O(n)$。为了克服单链表的这个缺点，引入了双链表，双链表结点中有两个指针 prior 和 next，分别指向其直接前驱和直接后继，如图 2.9 所示。表头结点的 prior 域和尾结点的 next 域都是 NULL。

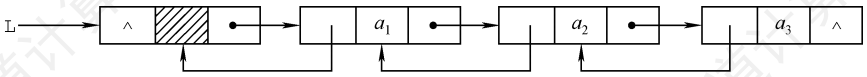


图 2.9　双链表示意图

双链表中结点类型的描述如下：

```
typedef struct DNode{                    //定义双链表结点类型
        ElemType data;                   //数据域
        struct DNode *prior,*next;       //前驱和后继指针
}DNode,*DLinklist;
```

双链表在单链表结点中增加了一个指向其前驱的指针 prior，因此双链表的按值查找和按位查找的操作与单链表的相同。但双链表在插入和删除操作的实现上，与单链表有着较大的不同。这是因为“链”变化时也需要对指针 prior 做出修改，其关键是保证在修改的过程中不断链。此外，双链表可以很方便地找到当前结点的前驱，因此，插入、删除操作的时间复杂度仅为 $O(1)$。

1．双链表的插入操作

在双链表中 p 所指的结点之后插入结点*s，其指针的变化过程如图 2.10 所示。

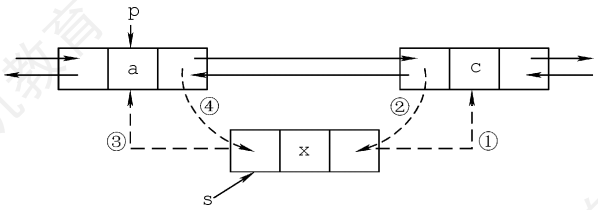


图 2.10　双链表插入结点过程

命题追踪　双链表中插入操作的实现（2023）

插入操作的代码片段如下：

```
① s->next=p->next;              //将结点*s 插入到结点*p 之后
② p->next->prior=s;
③ s->prior=p;
④ p->next=s;
```

上述代码的语句顺序不是唯一的，但也不是任意的，①步必须在④步之前，否则*p 的后继结点的指针就会丢掉，导致插入失败。为了加深理解，读者可以在纸上画出示意图。若问题改成要求在结点*p 之前插入结点*s，请读者思考具体的操作步骤。

配套课程公众号[考机研]
永久联系微信:4550060

2．双链表的删除操作

删除双链表中结点*p 的后继结点*q，其指针的变化过程如图 2.11 所示。

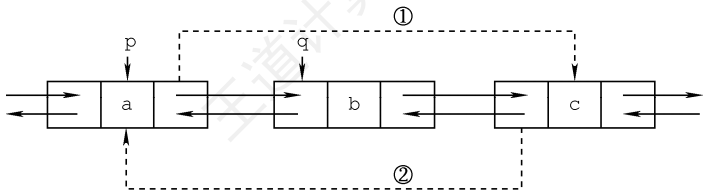


图 2.11　双链表删除结点过程

命题追踪 ▶ 双链表中删除操作的实现（2016）

删除操作的代码片段如下：

```
p->next=q->next;              //图 2.11 中步骤①
q->next->prior=p;             //图 2.11 中步骤②
free(q);                      //释放结点空间
```

若问题改成要求删除结点*q 的前驱结点*p，请读者思考具体的操作步骤。

在建立双链表的操作中，也可采用如同单链表的头插法和尾插法，但在操作上需要注意指针的变化和单链表有所不同。

2.3.4　循环链表

1．循环单链表

循环单链表和单链表的区别在于，表中最后一个结点的指针不是 NULL，而改为指向头结点，从而整个链表形成一个环，如图 2.12 所示。

在循环单链表中，表尾结点*r 的 next 域指向 L，故表中没有指针域为 NULL 的结点，因此，循环单链表的判空条件不是头结点的指针是否为空，而是它是否等于头指针 L。

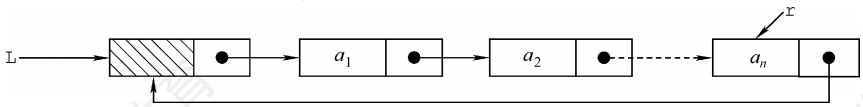


图 2.12　循环单链表

命题追踪 ▶ 循环单链表中删除首元素的操作（2021）

循环单链表的插入、删除算法与单链表的几乎一样，所不同的是，若操作是在表尾进行，则执行的操作不同，以让单链表继续保持循环的性质。当然，正是因为循环单链表是一个“环”，所以在任何位置上的插入和删除操作都是等价的，而无须判断是否是表尾。

在单链表中只能从表头结点开始往后顺序遍历整个链表，而循环单链表可以从表中的任意一个结点开始遍历整个链表。有时对循环单链表不设头指针而仅设尾指针，以使得操作效率更高。其原因是，若设的是头指针，对在表尾插入元素需要 $O(n)$的时间复杂度，而若设的是尾指针 r，r->next 即头指针，对在表头或表尾插入元素都只需要 $O(1)$的时间复杂度。

2．循环双链表

由循环单链表的定义不难推出循环双链表。不同的是，在循环双链表中，头结点的 prior 指针还要指向表尾结点，如图 2.13 所示。当某结点*p 为尾结点时，p->next==L；当循环双链表为空表时，其头结点的 prior 域和 next 域都等于 L。

配套课程公众号[考机研]
永久联系微信:4550060

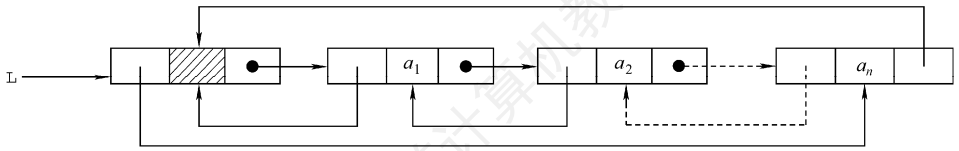
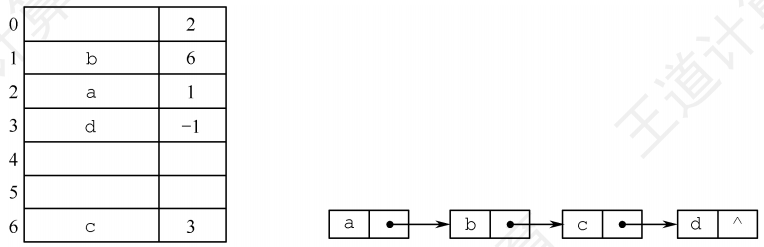


图 2.13　循环双链表

2.3.5　静态链表

静态链表是用数组来描述线性表的链式存储结构，结点也有数据域 data 和指针域 next，与前面所讲的链表中的指针不同的是，这里的指针是结点在数组中的相对地址（数组下标），也称游标。和顺序表一样，静态链表也要预先分配一块连续的内存空间。

静态链表和单链表的对应关系如图 2.14 所示。



(a)静态链表示例　　(b)静态链表对应的单链表

图 2.14　静态链表存储示意图

静态链表结构类型的描述如下：

```
#define MaxSize 50              //静态链表的最大长度
typedef struct{                 //静态链表结构类型的定义
    ElemType data;              //存储数据元素
    int next;                   //下一个元素的数组下标
} SLinkList[MaxSize];
```

静态链表以 next==-1 作为其结束的标志。静态链表的插入、删除操作与动态链表的相同，只需要修改指针，而不需要移动元素。总体来说，静态链表没有单链表使用起来方便，但在一些不支持指针的高级语言（如 Basic）中，这是一种非常巧妙的设计方法。

2.3.6　顺序表和链表的比较

1．存取（读/写）方式

顺序表既可以顺序存取，也可以随机存取，链表只能从表头开始依次顺序存取。例如在第 i 个位置上执行存取的操作，顺序表仅需一次访问，而链表则需从表头开始依次访问 i 次。

2．逻辑结构与物理结构

采用顺序存储时，逻辑上相邻的元素，对应的物理存储位置也相邻。而采用链式存储时，逻辑上相邻的元素，物理存储位置不一定相邻，对应的逻辑关系是通过指针链接来表示的。

3．查找、插入和删除操作

对于按值查找，顺序表无序时，两者的时间复杂度均为 $O(n)$；顺序表有序时，可采用折半查找，此时的时间复杂度为 $O(\log_2 n)$。对于按序号查找，顺序表支持随机访问，时间复杂度仅为 $O(1)$，而链表的平均时间复杂度为 $O(n)$。顺序表的插入、删除操作，平均需要移动半个表长的元素。链表的插入、删除操作，只需修改相关结点的指针域即可。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

4．空间分配

顺序存储在静态存储分配情形下，一旦存储空间装满就不能扩充，若再加入新元素，则会出现内存溢出，因此需要预先分配足够大的存储空间。预先分配过大，可能会导致顺序表后部大量闲置；预先分配过小，又会造成溢出。动态存储分配虽然存储空间可以扩充，但需要移动大量元素，导致操作效率降低，而且若内存中没有更大块的连续存储空间，则会导致分配失败。链式存储的结点空间只在需要时申请分配，只要内存有空间就可以分配，操作灵活、高效。此外，链表的每个结点都带有指针域，因此存储密度不够大。

在实际中应该怎样选取存储结构呢？

1．基于存储的考虑

难以估计线性表的长度或存储规模时，不宜采用顺序表；链表不用事先估计存储规模，但链表的存储密度较低，显然链式存储结构的存储密度是小于1的。

2．基于运算的考虑

在顺序表中按序号访问 a_i 的时间复杂度为 $O(1)$，而链表中按序号访问的时间复杂度为 $O(n)$，因此若经常做的运算是按序号访问数据元素，则显然顺序表优于链表。

在顺序表中进行插入、删除操作时，平均移动表中一半的元素，当数据元素的信息量较大且表较长时，这一点是不应忽视的；在链表中进行插入、删除操作时，虽然也要找插入位置，但操作主要是比较操作，从这个角度考虑显然后者优于前者。

3．基于环境的考虑

顺序表容易实现，任何高级语言中都有数组类型；链表的操作是基于指针的，相对来讲，前者实现较为简单，这也是用户考虑的一个因素。

总之，两种存储结构各有长短，选择哪一种由实际问题的主要因素决定。通常较稳定的线性表选择顺序存储，而频繁进行插入、删除操作的线性表（动态性较强）宜选择链式存储。

注　意

只有熟练掌握顺序存储和链式存储，才能深刻理解它们的优缺点。

2.3.7　本节试题精选

一、单项选择题

01. 下列关于线性表的存储结构的描述中，正确的是（　）。

I. 线性表的顺序存储结构优于其链式存储结构

II. 链式存储结构比顺序存储结构能更方便地表示各种逻辑结构

III. 若频繁使用插入和删除结点操作，则顺序存储结构更优于链式存储结构

IV. 顺序存储结构和链式存储结构都可以进行顺序存取

A. I、II、III　　B. II、IV　　C. II、III　　D. III、IV

02. 对于一个线性表，既要求能进行较快速地插入和删除，又要求存储结构能反映数据之间的逻辑关系，则应该用（　）。

A. 顺序存储方式　B. 链式存储方式　C. 散列存储方式　D. 以上均可以

03. 链式存储设计时，结点内的存储单元地址（　）。

A. 一定连续　　B. 一定不连续

C. 不一定连续　　D. 部分连续，部分不连续

配套课程公众号[考机研]
永久联系微信:4550060

04. 下列关于线性表的说法中，正确的是（　）。

I. 顺序存储方式只能用于存储线性结构

II. 在一个设有头指针和尾指针的单链表中，删除表尾元素的时间复杂度与表长无关

III. 带头结点的循环单链表中不存在空指针

IV. 在一个长度为 n 的有序单链表中插入一个新结点并仍保持有序的时间复杂度为 $O(n)$

V. 若用单链表来表示队列，则应该选用带尾指针的循环链表

A. I、II　　B. I、III、IV、V

C. IV、V　　D. III、IV、V

05. 设线性表中有 $2n$ 个元素，（　）在单链表上实现要比在顺序表上实现效率更高。

A. 删除所有值为 x 的元素

B. 在最后一个元素的后面插入一个新元素

C. 顺序输出前 k 个元素

D. 交换第 i 个元素和第 $2n-i-1$ 个元素的值（$i=0,\cdots,n-1$）

06. 在一个单链表中，已知 q 所指结点是 p 所指结点的前驱结点，若在 q 和 p 之间插入结点 s，则执行（　）。

A. `s->next=p->next;p->next=s;`　　B. `p->next=s->next;s->next=p;`

C. `q->next=s;s->next=p;`　　D. `p->next=s;s->next=q;`

07. 给定有 n 个元素的一维数组，建立一个有序单链表的最低时间复杂度是（　）。

A. $O(1)$　　B. $O(n)$　　C. $O(n^2)$　　D. $O(n\log_2 n)$

08. 将长度为 n 的单链表链接在长度为 m 的单链表后面，其算法的时间复杂度采用大 O 形式表示应该是（　）。

A. $O(1)$　　B. $O(n)$　　C. $O(m)$　　D. $O(n+m)$

09. 单链表中，增加一个头结点的目的是（　）。

A. 使单链表至少有一个结点　　B. 标识表结点中首结点的位置

C. 方便运算的实现　　D. 说明单链表是线性表的链式存储

10. 在一个长度为 n 的带头结点的单链表 h 上，设有尾指针 r，则执行（　）操作与链表的表长有关。

A. 删除单链表中的第一个元素

B. 删除单链表中的最后一个元素

C. 在单链表第一个元素前插入一个新元素

D. 在单链表最后一个元素后插入一个新元素

11. 对于一个头指针为 head 的带头结点的单链表，判定该表为空表的条件是（　）；对于不带头结点的单链表，判定空表的条件为（　）。

A. `head==NULL`　　B. `head->next==NULL`

C. `head->next==head`　　D. `head!=NULL`

12. 在线性表 $a_0, a_1, \cdots, a_{100}$ 中，删除元素 a_{50} 需要移动（　）个元素。

A. 0　　B. 50　　C. 51　　D. 0 或 50

13. 通过含有 $n(n>1)$ 个元素的数组 a，采用头插法建立单链表 L，则 L 中的元素次序（　）。

A. 与数组 a 的元素次序相同　　B. 与数组 a 的元素次序相反

C. 与数组 a 的元素次序无关　　D. 以上都错误

14. 下面关于线性表的一些说法中，正确的是（　）。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

A. 对一个设有头指针和尾指针的单链表执行删除最后一个元素的操作与链表长度无关
B. 线性表中每个元素都有一个直接前驱和一个直接后继
C. 为了方便插入和删除数据，可以使用双链表存放数据
D. 取线性表第 i 个元素的时间与 i 的大小有关

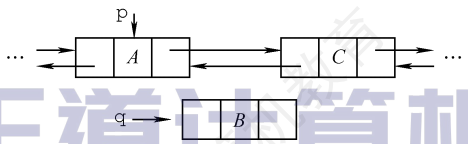
15. 在双链表中向 p 所指的结点之前插入一个结点 q 的操作为（ ）。

A. `p->prior=q; q->next=p; p->prior->next=q; q->prior=p->prior;`
B. `q->prior=p->prior; p->prior->next=q; q->next=p; p->prior=q->next;`
C. `q->next=p; p->next=q; q->prior->next=q; q->next=p;`
D. `p->prior->next=q; q->next=p; q->prior=p->prior; p->prior=q;`

16. 在双链表存储结构中，删除 p 所指的结点时必须修改指针（ ）。

A. `p->prior->next=p->next; p->next->prior=p->prior;`
B. `p->prior=p->prior->prior; p->prior->next=p;`
C. `p->next->prior=p; p->next=p->next->next;`
D. `p->next=p->prior->prior; p->prior=p->next->next;`

17. 在如下图所示的双链表中，已知指针 p 指向结点 A，若要在结点 A 和 C 之间插入指针 q 所指的结点 B，则依次执行的语句序列可以是（ ）。



①`q->next=p->next;` ②`q->prior=p;` ③ `p->next=q;` ④ `p->next->prior=q;`

A. ①②④③ B. ④③②① C. ③④①② D. ①③④②

18. 在双链表的两个结点之间插入一个新结点，需要修改（ ）个指针域。

A. 1 B. 3 C. 4 D. 2

19. 在长度为 n 的有序单链表中插入一个新结点，并仍然保持有序的时间复杂度是（ ）。

A. $O(1)$ B. $O(n)$ C. $O(n^2)$ D. $O(n\log_2 n)$

20. 与单链表相比，双链表的优点之一是（ ）。

A. 插入、删除操作更方便 B. 可以进行随机访问
C. 可以省略表头指针或表尾指针 D. 访问前后相邻结点更灵活

21. 对于一个带头结点的循环单链表 L，判断该表为空表的条件是（ ）。

A. 头结点的指针域为空 B. L 的值为 NULL
C. 头结点的指针域与 L 的值相等 D. 头结点的指针域与 L 的地址相等

22. 对于一个带头结点的循环双链表 L，判断该表为空表的条件是（ ）。

A. `L->prior==L&&L->next==NULL` B. `L->prior==NULL&&L->next==NULL`
C. `L->prior==NULL&&L->next==L` D. `L->prior==L&&L->next==L`

23. 一个链表最常用的操作是在末尾插入结点和删除结点，则选用（ ）最节省时间。

A. 带头结点的循环双链表 B. 循环单链表
C. 带尾指针的循环单链表 D. 单链表

24. 设对 n（$n>1$）个元素的线性表的运算只有 4 种：删除第一个元素；删除最后一个元素；在第一个元素之前插入新元素；在最后一个元素之后插入新元素，则最好使用（ ）。

A. 只有尾结点指针没有头结点指针的循环单链表

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

B. 只有尾结点指针没有头结点指针的非循环双链表
C. 只有头结点指针没有尾结点指针的循环双链表
D. 既有头结点指针又有尾结点指针的循环单链表

25. 设有两个长度为 n 的循环单链表，若要求两个循环单链表的头尾相接的时间复杂度为 $O(1)$，则对应两个循环单链表各设置一个指针，分别指向（　）。
A. 各自的头结点　　B. 各自的尾结点
C. 各自的首结点　　D. 一个表的头结点，另一个表的尾结点

26. 设有一个长度为 n 的循环单链表，若从表中删除首元结点的时间复杂度达到 $O(n)$，则此时采用的循环单链表的结构可能是（　）。
A. 只有表头指针，没有头结点　　B. 只有表尾指针，没有头结点
C. 只有表尾指针，带头结点　　D. 只有表头指针，带头结点

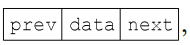
27. 某线性表用带头结点的循环单链表存储，头指针为 head，当 head->next-> next== head 成立时，线性表的长度可能是（　）。
A. 0　　B. 1　　C. 2　　D. 可能为 0 或 1

28. 有两个长度都为 n 的双链表，若以 h_1 为头指针的双链表是非循环的，以 h_2 为头指针的双链表是循环的，则下列叙述中正确的是（　）。
A. 对于双链表 h_1，删除首结点的时间复杂度是 $O(n)$
B. 对于双链表 h_2，删除首结点的时间复杂度是 $O(n)$
C. 对于双链表 h_1，删除尾结点的时间复杂度是 $O(1)$
D. 对于双链表 h_2，删除尾结点的时间复杂度是 $O(1)$

29. 一个链表最常用的操作是在最后一个元素后插入一个元素和删除第一个元素，则选用（　）最节省时间。
A. 不带头结点的循环单链表　　B. 双链表
C. 单链表　　D. 不带头结点且有尾指针的循环单链表

30. 需要分配较大空间，插入和删除不需要移动元素的线性表，其存储结构为（　）。
A. 单链表　　B. 静态链表　　C. 顺序表　　D. 双链表

31. 下列关于静态链表的说法中，正确的是（　）。
I. 静态链表兼具顺序表和单链表的优点，因此存取表中第 i 个元素的时间与 i 无关
II. 静态链表能容纳的最大元素个数在表定义时就确定了，以后不能增加
III. 静态链表与动态链表在元素的插入、删除上类似，不需要移动元素
IV. 相比动态链表，静态链表可能浪费较多的存储空间
A. I、II、III　　B. II、III、IV
C. I、III、IV　　D. I、II、IV

32. 【2016 统考真题】已知一个带有表头结点的循环双链表 L，结点结构为 

，其中 prev 和 next 分别是指向其直接前驱和直接后继结点的指针。现要删除指针 p 所指的结点，正确的语句序列是（　）。
A. `p->next->prev=p->prev; p->prev->next=p->prev; free(p);`
B. `p->next->prev=p->next; p->prev->next=p->next; free(p);`
C. `p->next->prev=p->next; p->prev->next=p->prev; free(p);`
D. `p->next->prev=p->prev; p->prev->next=p->next; free(p);`

33. 【2016 统考真题】已知表头元素为 c 的单链表在内存中的存储状态如下表所示。

配套课程公众号[考机研]
永久联系微信:4550060

地址	元素	链接地址
1000H	a	1010H
1004H	b	100CH
1008H	c	1000H
100CH	d	NULL
1010H	e	1004H
1014H		

现将f存放于1014H处并插入单链表，若f在逻辑上位于a和e之间，则a、e、f的“链接地址”依次是（ ）。

A. 1010H、1014H、1004H　　B. 1010H、1004H、1014H

C. 1014H、1010H、1004H　　D. 1014H、1004H、1010H

34. 【2021统考真题】已知头指针h指向一个带头结点的非空循环单链表，结点结构为 data | next，其中next是指向直接后继结点的指针，p是尾指针，q是临时指针。现要删除该链表的第一个元素，正确的语句序列是（ ）。

A. `h->next=h->next->next;q=h->next;free(q);`

B. `q=h->next;h->next=h->next->next;free(q);`

C. `q=h->next;h->next=q->next;if(p!=q) p=h;free(q);`

D. `q=h->next;h->next=q->next;if(p==q) p=h;free(q);`

35. 【2023统考真题】现有非空双链表L，其结点结构为 prev | data | next，prev是指向直接前驱结点的指针，next是指向直接后继结点的指针。若要在L中指针p所指向的结点（非尾结点）之后插入指针s指向的新结点，则在执行语句序列“`s->next= p->next; p->next=s;`”后，下列语句序列中还需要执行的是（ ）。

A. `s->next->prev=p; s->prev=p;`

B. `p->next->prev=s; s->prev=p;`

C. `s->prev=s->next->prev; s->next->prev=s;`

D. `p->next->prev=s->prev; s->next->prev=p;`

36. 【2024统考真题】已知带头结点的非空单链表L的头指针为h，结点结构为 data | next，其中next是指向直接后继结点的指针。现有指针p和q，若p指向L中非首且非尾的任意一个结点，则执行语句序列“`q=p->next; p->next=q->next; q->next=h->next; h->next=q;`”的结果是（ ）。

A. 在p所指结点后插入q所指结点　　B. 在q所指结点后插入p所指结点

C. 将p所指结点移至L的头结点之后　　D. 将q所指结点移动到L的头结点之后

二、综合应用题

01. 在带头结点的单链表L中，删除所有值为x的结点，并释放其空间，假设值为x的结点不唯一，试编写算法以实现上述操作。

02. 试编写在带头结点的单链表L中删除一个最小值结点的高效算法（假设该结点唯一）。

03. 试编写算法将带头结点的单链表就地逆置，所谓“就地”是指辅助空间复杂度为$O(1)$。

04. 设在一个带表头结点的单链表中，所有结点的元素值无序，试编写一个函数，删除表中所有处于给定的两个值（作为函数参数给出）之间的元素（若存在）。

05. 给定两个单链表，试分析找出两个链表的公共结点的思想（不用写代码）。

06. 设$C=\{a_1,b_1,a_2,b_2,\cdots,a_n,b_n\}$为线性表，采用带头结点的单链表存放，设计一个就地算法，将其拆分为两个线性表，使得$A=\{a_1,a_2,\cdots,a_n\}$，$B=\{b_n,\cdots,b_2,b_1\}$。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

07. 在一个递增有序的单链表中，存在重复的元素。设计算法删除重复的元素，例如 (7, 10, 10, 21, 30, 42, 42, 42, 51, 70) 将变为 (7, 10, 21, 30, 42, 51, 70)。

08. 设 A 和 B 是两个单链表 (带头结点)，其中元素递增有序。设计一个算法从 A 和 B 中的公共元素产生单链表 C，要求不破坏 A、B 的结点。

09. 已知两个链表 A 和 B 分别表示两个集合，其元素递增排列。编制函数，求 A 与 B 的交集，并存放于 A 链表中。

10. 两个整数序列 $A=a_1, a_2, a_3, \cdots, a_m$ 和 $B=b_1, b_2, b_3, \cdots, b_n$ 已经存入两个单链表中，设计一个算法，判断序列 B 是否是序列 A 的连续子序列。

11. 设计一个算法用于判断带头结点的循环双链表是否对称。

12. 有两个循环单链表，链表头指针分别为 h1 和 h2，编写一个函数将链表 h2 链接到链表 h1 之后，要求链接后的链表仍保持循环链表形式。

13. 设有一个带头结点的非循环双链表 L，其每个结点中除有 pre、data 和 next 域外，还有一个访问频度域 freq，其值均初始化为零。每当在链表中进行一次 Locate(L,x) 运算时，令值为 x 的结点中 freq 域的值增 1，并使此链表中的结点保持按访问频度递减的顺序排列，且最近访问的结点排在频度相同的结点之前，以便使频繁访问的结点总是靠近表头。试编写符合上述要求的 Locate(L,x) 函数，返回找到结点的地址，类型为指针型。

14. 设将 n ($n>1$) 个整数存放到不带头结点的单链表 L 中，设计算法将 L 中保存的序列循环右移 k ($0<k<n$) 个位置。例如，若 $k=1$，则将链表{0, 1, 2, 3}变为{3, 0, 1, 2}。要求：

1）给出算法的基本设计思想。

2）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。

15. 单链表有环，是指单链表的最后一个结点的指针指向了链表中的某个结点 (通常单链表的最后一个结点的指针域是空的)。试编写算法判断单链表是否存在环。

1）给出算法的基本设计思想。

2）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度和空间复杂度。

16. 设有一个长度 n (n 为偶数) 的不带头结点的单链表，且结点值都大于 0，设计算法求这个单链表的最大孪生和。孪生和定义为一个结点值与其孪生结点值之和，对于第 i 个结点 (从 0 开始)，其孪生结点为第 $n-i-1$ 个结点。要求：

1）给出算法的基本设计思想。

2）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

3）说明你的算法的时间复杂度和空间复杂度。

17. 【2009 统考真题】已知一个带有表头结点的单链表，结点结构为

data	link

假设该链表只给出了头指针 list。在不改变链表的前提下，请设计一个尽可能高效的算法，查找链表中倒数第 k 个位置上的结点 (k 为正整数)。若查找成功，算法输出该结点的 data 域的值，并返回 1；否则，只返回 0。要求：

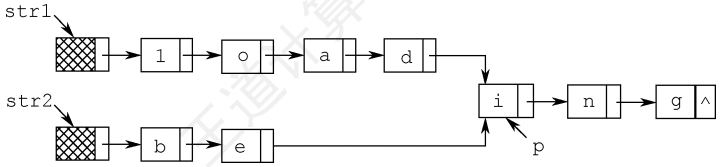
1）描述算法的基本设计思想。

2）描述算法的详细实现步骤。

3）根据设计思想和实现步骤，采用程序设计语言描述算法 (使用 C、C++或 Java 语言实现)，关键之处请给出简要注释。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

18. 【2012 统考真题】假定采用带头结点的单链表保存单词，当两个单词有相同的后缀时，可共享相同的后缀存储空间，例如，loading 和 being 的存储映像如下图所示。



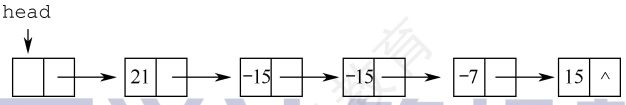
设 str1 和 str2 分别指向两个单词所在单链表的头结点，链表结点结构为 data | next，请设计一个时间上尽可能高效的算法，找出由 str1 和 str2 所指向两个链表共同后缀的起始位置（如图中字符 i 所在结点的位置 p）。要求：

1）给出算法的基本设计思想。

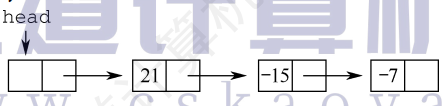
2）根据设计思想，采用 C 或 C++或 Java 语言描述算法，关键之处给出注释。

3）说明你所设计算法的时间复杂度。

19. 【2015 统考真题】用单链表保存 m 个整数，结点的结构为 [data][link]，且|data|≤n（n 为正整数）。现要求设计一个时间复杂度尽可能高效的算法，对于链表中 data 的绝对值相等的结点，仅保留第一次出现的结点而删除其余绝对值相等的结点。例如，若给定的单链表 head 如下：



则删除结点后的 head 为



要求：

1）给出算法的基本设计思想。

2）使用 C 或 C++语言，给出单链表结点的数据类型定义。

3）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

4）说明你所设计算法的时间复杂度和空间复杂度。

20. 【2019 统考真题】设线性表 $L=(a_1,a_2,a_3,\cdots,a_{n-2},a_{n-1},a_n)$ 采用带头结点的单链表保存，链表中的结点定义如下：

```
typedef struct node
{   int data;
    struct node*next;
}NODE;
```

请设计一个空间复杂度为 $O(1)$且时间上尽可能高效的算法，重新排列 L 中的各结点，得到线性表 $L'=(a_1,a_n,a_2,a_{n-1},a_3,a_{n-2},\cdots)$。要求：

1）给出算法的基本设计思想。

2）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

3）说明你所设计的算法的时间复杂度。

2.3.8 答案与解析

一、单项选择题

01. B

两种存储结构适用于不同的场合，不能简单地说谁好谁坏，选项 I 错误。链式存储用指针表

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

示逻辑结构，而指针的设置是任意的，因此比顺序存储结构能更方便地表示各种逻辑结构，选项 II 正确。在顺序存储中，插入和删除结点需要移动大量元素，效率较低，选项 III 的描述刚好相反。顺序存储结构既能随机存取又能顺序存取，而链式结构只能顺序存取，选项 IV 正确。

02．B

首先直接排除选项 A 和 D。散列存储通过散列函数映射到物理空间，不能反映数据之间的逻辑关系，排除选项 C。链式存储能方便地表示各种逻辑关系，且插入和删除操作的时间复杂度为 $O(1)$。

03．A

链式存储设计时，各个不同结点的存储空间可以不连续，但结点内的存储单元地址必须连续。

04．D

顺序存储方式同样适用于存储图和树，选项 I 错误。删除表尾结点时，必须从头开始找到表尾结点的前驱，其时间与表长有关，选项 II 错误。循环单链表中最后一个结点的指针不是 NULL，而是指向头结点，整个链表形成一个环，因此不存在空指针，选项 III 正确。有序单链表只能依次查找插入位置，时间复杂度为 $O(n)$，选项 IV 正确。队列需要在表头删除元素，表尾插入元素，采用带尾指针的循环链表较为方便，插入和删除的时间复杂度都为 $O(1)$，选项 V 正确。

05．A

对于选项 A，在单链表和顺序表上实现的时间复杂度都为 $O(n)$，但后者要移动很多元素，因此在单链表上实现效率更高。对于选项 B 和 D，顺序表的效率更高。C 无区别。

06．C

s 插入后，q 成为 s 的前驱，而 p 成为 s 的后继，选择选项 C。

注　意

可能有读者认为选项 C 中的两条语句交换后才正确。实际上，因为本题插入位置的前后结点都有指针指示（这与前面介绍的插入操作是不同的），所以选项 C 中的语句顺序并不会造成断链。在此提醒读者在学习过程中一定要多动脑思考，而不要生搬硬套。

07．D

若先建立链表，然后依次插入建立有序表，则每插入一个元素就需遍历链表寻找插入位置，即直接插入排序，时间复杂度为 $O(n^2)$。若先将数组排好序，然后建立链表，建立链表的时间复杂度为 $O(n)$，数组排序的最好时间复杂度为 $O(n\log_2 n)$，总时间复杂度为 $O(n\log_2 n)$。故选择选项 D。

08．C

先遍历长度为 m 的单链表，找到该单链表的尾结点，然后将其 next 域指向另一个单链表的首结点，其时间复杂度为 $O(m)$。

09．C

单链表设置头结点的目的是方便运算的实现，主要好处体现在：第一，有头结点后，插入和删除数据元素的算法就统一了，不再需要判断是否在第一个元素之前插入或删除第一个元素；第二，不论链表是否为空，其头指针是指向头结点的非空指针，链表的头指针不变，因此空表和非空表的处理也就统一了。

10．B

删除单链表的最后一个结点需置其前驱结点的指针域为 NULL，需要从头开始依次遍历找到该前驱结点，需要 $O(n)$的时间，与表长有关。其他操作均与表长无关，读者可自行模拟。

11．B，A

在带头结点的单链表中，头指针 head 指向头结点，头结点的 next 域指向第一个元素结点，head->next==NULL 表示该单链表为空。在不带头结点的单链表中，head 直接指向第一个元

配套课程公众号[考机研]
永久联系微信:4550060

素结点，head==NULL 表示该单链表为空。

12. D

线性表有顺序存储和链式存储两种存储结构。若采用链式存储结构，则删除元素 a_{50} 不需要移动元素；若采用顺序存储结构，则需要依次移动 50 个元素。

13. B

当采用头插法建立单链表时，数组后面的元素插入到单链表 L 的最前端，所以 L 中的元素次序与数组 a 的元素次序相反。

14. C

选项 A 显然错误。选项 B 中第一个元素和最后一个元素不满足题设要求。双链表能很方便地访问前驱和后继，故删除和插入数据较为方便，选项 C 正确。选项 D 未考虑顺序存储的情况。

15. D

为了在 p 之前插入结点 q，可以将 p 的前一个结点的 next 域指向 q，将 q 的 next 域指向 p，将 q 的 prior 域指向 p 的前一个结点，将 p 的 prior 域指向 q。仅 D 满足条件。

16. A

与上一题的分析基本类似，只不过这里是删除一个结点，注意将 p 的前、后两结点链接起来。关键是要保证在结点指针的修改过程中不断链！

注意，请读者仔细对比上述两题，弄清双链表的插入和删除方法。

17. A

结点 A 和 B 分别由指针 p 和 q 指示，但结点 C 仅能由 p->next 间接指示，因此在改变 p->next 之前，必须先将 q->next 指向结点 C，即①要在③前面，且④要在③前面（因为若先执行③，则④相当于 q->prior 指向其自身，显然矛盾）。故只能选择选项 A。

18. C

当在双链表的两个结点（分别用第一个、第二个结点表示）之间插入一个新结点时，需要修改四个指针域，分别是：新结点的前驱指针域，指向第一个结点；新结点的后继指针域，指向第二个结点；第一个结点的后继指针域，指向新结点；第二个结点的前驱指针域，指向新结点。

19. B

设单链表递增有序，首先要在单链表中找到第一个大于 x 的结点的直接前驱 p，在 p 之后插入该结点。查找的时间复杂度为 $O(n)$，插入的时间复杂度为 $O(1)$，总时间复杂度为 $O(n)$。

20. D

在插入和删除操作上，单链表和双链表都不用移动元素，都很方便，但双链表修改指针的操作更为复杂，选项 A 错误。双链表中可以快速访问任何一个结点的前驱和后继结点，选项 D 正确。

21. C

带头结点的循环单链表 L 为空表时，满足 L->next==L，即头结点的指针域与 L 的值相等，而不是头结点的指针域与 L 的地址相等。注意，带头结点的循环单链表中不存在空指针。

22. D

循环双链表 L 判空的条件是头结点（头指针）的 prior 和 next 域都指向它自身。

23. A

在链表的末尾插入和删除一个结点时，需要修改其相邻结点的指针域。而寻找尾结点及尾结点的前驱结点时，只有带头结点的循环双链表所需要的时间最少。

24. C

对于选项 A，删除尾结点*p 时，需要找到*p 的前一个结点，时间复杂度为 $O(n)$。对于选项 B，删除首结点*p 时，需要找到*p 结点，这里没有直接给出头结点指针，而通过尾结点的 prior

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

指针找到*p 结点的时间复杂度为 $O(n)$。对于选项 D，删除尾结点*p 时，需要找到*p 的前一个结点，时间复杂度为 $O(n)$。对于 C，执行这四种算法的时间复杂度均为 $O(1)$。

25．B

要求用 $O(1)$的时间将两个循环单链表头尾相接，并未指明哪个链表接在另一个链表之后，所以对两个链表都要在 $O(1)$的时间找到头结点和尾结点。因此，两个指针应都指向尾结点。

26．A

在循环单链表中，删除首元结点后，要保持链表的循环性，因此需要找到首元结点的前驱。当链表带头结点时，其前驱就是头结点，因此不论是表头指针还是表尾指针，删除首元结点的时间都为 $O(1)$。当链表不带头结点时，其前驱是尾结点，因此，若有表尾指针，就可在 $O(1)$的时间找到尾结点；若只有表头指针，则需要遍历整个链表找到尾结点，时间为 $O(n)$。

27．D

对一个空循环单链表，有 `head->next==head`，推理 `head->next->next==head->next==head`。对含有一个元素的循环单链表，头结点（头指针 `head` 指示）的 `next` 域指向这个唯一的元素结点，该元素结点的 `next` 域指向头结点，因此也有 `head->next->next=head`。

28．D

对于两种双链表，删除首结点的时间复杂度都是 $O(1)$。对于非循环双链表，删除尾结点的时间复杂度是 $O(n)$；对于循环双链表，删除尾结点的时间复杂度是 $O(1)$。

29．D

对于选项 A，在最后一个元素之后插入元素的情况与普通单链表相同，时间复杂度为 $O(n)$；而删除第一个元素时，为保持循环单链表的性质（尾结点指向第一个结点），要先遍历整个链表找到尾结点，再做删除操作，时间复杂度为 $O(n)$。对于选项 B，双链表的情况与单链表的相同，一个是 $O(n)$，一个是 $O(1)$。对于选项 C，在最后一个元素之后插入一个元素，要遍历整个链表才能找到插入位置，时间复杂度为 $O(n)$；删除第一个元素的时间复杂度为 $O(1)$。对于选项 D，与选项 A 的分析对比，有尾结点的指针，省去了遍历链表的过程，因此时间复杂度均为 $O(1)$。

30．B

静态链表采用数组表示，因此需要预先分配较大的连续空间，静态链表同时还具有一般链表的特点，即插入和删除不需要移动元素。

31．B

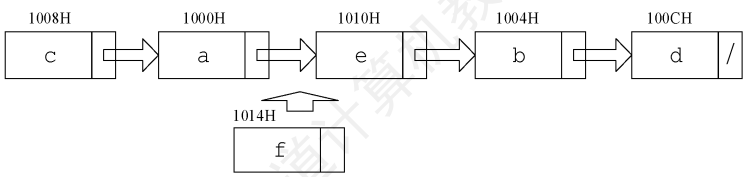
静态链表的存储空间虽然是顺序分配的，但元素的存储不是顺序的，查找时仍然需要按链依次进行，而插入、删除都不需要移动元素。静态链表的存储空间是一次性申请的，能容纳的最大元素个数在定义时就已确定。并非每个空间都存储了元素，因此会造成存储空间的浪费。

32．D

选项 A 的第二句代码，相当于将 p 前驱结点的后继指针指向其自身，错误；选项 B 和 C 的第一句代码，相当于将 p 后继结点的前驱指针指向其自身，错误。只有选项 D 正确。

33．D

根据存储状态，单链表的结构如下图所示。



配套课程公众号[考机研]
永久联系微信:4550060

其中“链接地址”是指结点 next 所指的内存地址。当结点 f 插入后，a 指向 f，f 指向 e，e 指向 b。显然 a、e 和 f 的“链接地址”分别是 f、b 和 e 的内存地址，即 1014H、1004H 和 1010H。

34．D

如图 1 所示，要删除带头结点的非空循环单链表中的第一个元素，就要先用临时指针 q 指向待删结点，q=h->next；然后将 q 从链表中断开，h->next=q->next（这一步也可写成 h->next=h->next->next）；此时要考虑一种特殊情况，若待删结点是链表的尾结点，即循环单链表中只有一个元素(p和q指向同一个结点)，如图2所示，则在删除后要将尾指针指向头结点，即 if (p==q) p=h；最后释放 q 结点即可，答案选择选项 D。

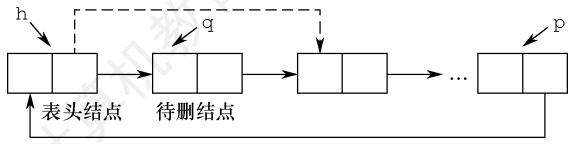


图 1

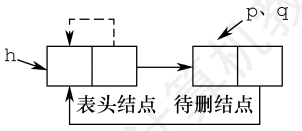
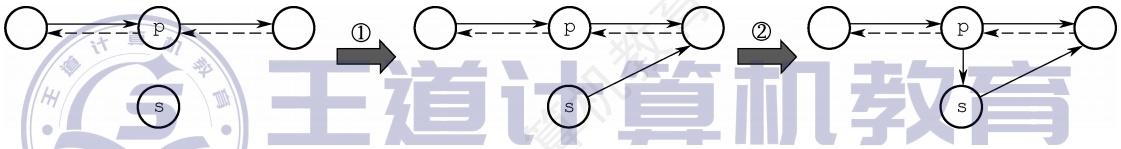


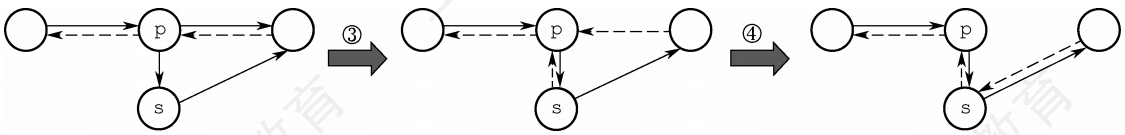
图 2

35．C

链表的插入操作要保证不会造成断链，画图再依次判断选项。执行完语句“①s->next=p->next；②p->next=s；”后的结构如下图所示（虚线表示 prev，实线表示 next）。

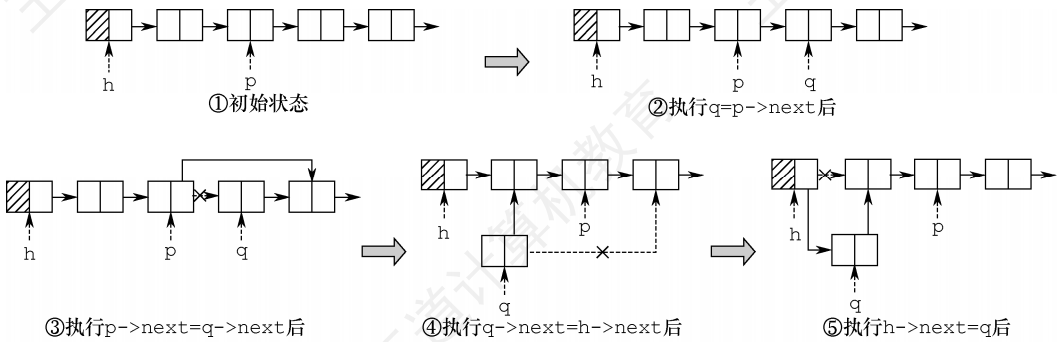


对于选项 A，s->next->prev=p，错误。对于选项 B，p->next->prev=s，让 s 的 prev 指向 s，错误。对于选项 D，两句代码均错误。对于选项 C，执行完语句“③s->prev=s->next->prev；④s->next->prev=s；”后的结构如下图所示，满足插入的要求。



36．D

假设单链表 L 的初始状态如图①所示。执行 q=p->next 后的状态如图②所示，q 指向 p 的后继结点。执行 p->next=q->next 后的状态如图③所示，结点 p 的 next 指向 q 的后继结点。执行 q->next=h->next 后的状态如图④所示，结点 q 的 next 指向 h 的后继结点。执行 h->next=q 后的状态如图⑤所示，q 所指结点移至 L 的头结点 h 之后，选项 D 正确。



配套课程公众号[考机研]
永久联系微信:4550060

二、综合应用题

01.【解答】

解法 1：用 p 从头至尾扫描单链表，pre 指向*p 结点的前驱。若 p 所指结点的值为 x，则删除，并让 p 移向下一个结点，否则让 pre、p 指针同步后移一个结点。

本题代码如下：

```
void Del_X_1(Linklist &L,ElemType x){
    LNode *p=L->next,*pre=L,*q;//置 p 和 pre 的初始值
    while(p!=NULL){
        if(p->data==x){
            q=p;                    //q 指向被删结点
            p=p->next;
            pre->next=p;            //将*q 结点从链表中断开
            free(q);                //释放*q 结点的空间
        }
        else{                       //否则，pre 和 p 同步后移
            pre=p;
            p=p->next;
        }//else
    }//while
}
```

本算法是在无序单链表中删除满足某种条件的所有结点，这里的条件是结点的值为 x。实际上，这个条件是可以任意指定的，只要修改 if 条件即可。比如，我们要求删除值介于 mink 和 maxk 之间的所有结点，则只需将 if 语句修改为 if(p->data>mink&&p->data<maxk)。

解法 2：采用尾插法建立单链表。用 p 指针扫描 L 的所有结点，当其值不为 x 时，将其链接到 L 之后，否则将其释放。

本题代码如下：

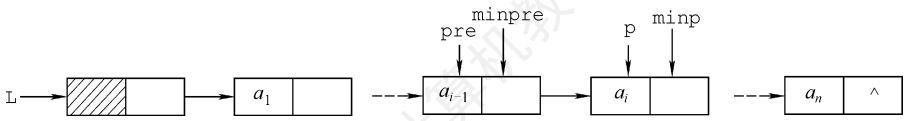
```
void Del_X_2(Linklist &L,ElemType x){
    LNode *p=L->next,*r=L,*q;   //r 指向尾结点，其初值为头结点
    while(p!=NULL){
        if(p->data!=x){         //*p 结点值不为 x 时将其链接到 L 尾部
            r->next=p;
            r=p;
            p=p->next;          //继续扫描
        }
        else{                   //*p 结点值为 x 时将其释放
            q=p;
            p=p->next;          //继续扫描
            free(q);            //释放空间
        }
    }//while
    r->next=NULL;               //插入结束后置尾结点指针为 NULL
}
```

上述两个算法扫描一遍链表，时间复杂度为 $O(n)$，空间复杂度为 $O(1)$。

02.【解答】

算法思想：用 p 从头至尾扫描单链表，pre 指向*p 结点的前驱，用 minp 保存值最小的结点指针（初值为 p），minpre 指向*minp 结点的前驱（初值为 pre）。一边扫描，一边比较，若 p->data 小于 minp->data，则将 p、pre 分别赋值给 minp、minpre，如下图所示。当 p 扫描完毕时，minp 指向最小值结点，minpre 指向最小值结点的前驱结点，再将 minp 所指结点删除即可。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060



本题代码如下：

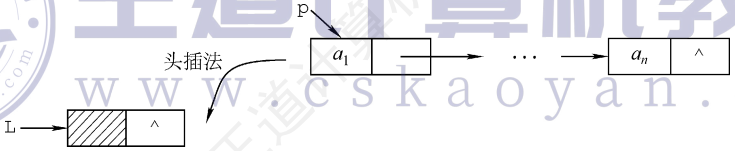
```
LinkList Delete_Min(LinkList &L){
    LNode *pre=L,*p=pre->next; //p为工作指针，pre指向其前驱
    LNode *minpre=pre,*minp=p; //保存最小值结点及其前驱
    while(p!=NULL){
        if(p->data<minp->data){
            minp=p;              //找到比之前找到的最小值结点更小的结点
            minpre=pre;
        }
        pre=p;                   //继续扫描下一个结点
        p=p->next;
    }
    minpre->next=minp->next;     //删除最小值结点
    free(minp);
    return  L;
}
```

算法需要从头至尾扫描链表，时间复杂度为 $O(n)$，空间复杂度为 $O(1)$。

若本题改为不带头结点的单链表，则实现上会有所不同，请读者自行思考。

03.【解答】

解法 1：将头结点摘下，然后从第一结点开始，依次插入到头结点的后面（头插法建立单链表），直到最后一个结点为止，这样就实现了链表的逆置，如下图所示。



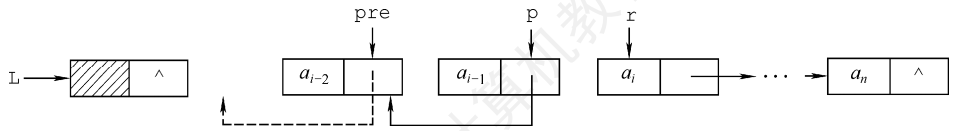
本题代码如下：

```
LinkList Reverse_1(LinkList L){
    LNode *p, *r;                //p为工作指针，r为p的后继，以防断链
    p=L->next;                   //从第一个元素结点开始
    L->next=NULL;                //先将头结点L的next域置为NULL
    while(p!=NULL){              //依次将元素结点摘下
        r=p->next;               //暂存p的后继
        p->next=L->next;         //将p结点插入到头结点之后
        L->next=p;
        p=r;
    }
    return  L;
}
```

解法 2：大部分辅导书都只介绍解法 1，这对读者的理解和思维是不利的。为了将调整指针这个复杂的过程分析清楚，我们借助图形来进行直观的分析。

假设 pre、p 和 r 指向三个相邻的结点，如下图所示。假设经过若干操作后，*pre 之前的结点的指针都已调整完毕，它们的 next 都指向其原前驱结点。现在令*p 结点的 next 域指向*pre 结点，注意到一旦调整指针的指向，*p 的后继结点的链就会断开，为此需要用 r 来指向原*p 的后继结点。处理时需要注意两点：一是在处理第一个结点时，应将其 next 域置为 NULL，而不是指向头结点（因为它将作为新表的尾结点）；二是在处理完最后一个结点后，需要将头结点的指针指向它。

配套课程公众号[考机研]
永久联系微信:4550060



本题代码如下：

```
LinkList Reverse_2(LinkList L){
    LNode *pre,*p=L->next,*r=p->next;
    p->next=NULL;                    //处理第一个结点
    while(r!=NULL){                  //r 为空，则说明 p 为最后一个结点
        pre=p;                       //依次继续遍历
        p=r;
        r=r->next;
        p->next=pre;                 //指针反转
    }
    L->next=p;                       //处理最后一个结点
    return  L;
}
```

上述两个算法的时间复杂度为 $O(n)$，空间复杂度为 $O(1)$。

04.【解答】

因为链表是无序的，所以只能逐个结点进行检查，执行删除。

本题代码如下：

```
void RangeDelete(LinkList &L,int min,int max){
    LNode *pr=L,*p=L->link;              //p 是检测指针，pr 是其前驱
    while(p!=NULL)
        if(p->data>min&&p->data<max){    //寻找到被删结点，删除
            pr->link=p->link;
            free(p);
            p=pr->link;
        }
        else{                            //否则继续寻找被删结点
            pr=p;
            p=p->link;
        }
}
```

05.【解答】

两个单链表有公共结点，即两个链表从某一结点开始，它们的 next 都指向同一结点。每个单链表结点只有一个 next 域，因此从第一个公共结点开始，之后的所有结点都是重合的，不可能再出现分叉。所以两个有公共结点而部分重合的单链表，拓扑形状看起来像 Y，而不可能像 X。

本题极容易联想到“蛮”方法：在第一个链表上顺序遍历每个结点，每遍历一个结点，在第二个链表上顺序遍历所有结点，若找到两个相同的结点，则找到了它们的公共结点。显然，该算法的时间复杂度为 $O(\text{len1}\times\text{len2})$。

接下来我们试着去寻找一个线性时间复杂度的算法。先把问题简化：如何判断两个单链表有没有公共结点？应注意到这样一个事实：若两个链表有一个公共结点，则该公共结点之后的所有结点都是重合的，即它们的最后一个结点必然是重合的。因此，我们判断两个链表是不是有重合的部分时，只需要分别遍历两个链表到最后一个结点。若两个尾结点是一样的，则说明它们有公共结点，否则两个链表没有公共结点。

然而，在上面的思路中，顺序遍历两个链表到尾结点时，并不能保证在两个链表上同时到达尾结点。这是因为两个链表长度不一定一样。但假设一个链表比另一个长 k 个结点，我们先在长的链表上遍历 k 个结点，之后再同步遍历，此时我们就能保证同时到达最后一个结点。两个链表

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

从第一个公共结点开始到链表的尾结点，这一部分是重合的，因此它们肯定也是同时到达第一公共结点的。于是在遍历中，第一个相同的结点就是第一个公共的结点。

根据这一思路中，我们先要分别遍历两个链表得到它们的长度，并求出两个长度之差。在长的链表上先遍历长度之差个结点之后，再同步遍历两个链表，直到找到相同的结点，或者一直到链表结束。此时，该方法的时间复杂度为$O(len1 + len2)$。

06.【解答】

算法思想：循环遍历链表C，采用尾插法将一个结点插入表A，这个结点为奇数号结点，这样建立的表A与原来的结点顺序相同；采用头插法将下一结点插入表B，这个结点为偶数号结点，这样建立的表B与原来的结点顺序正好相反。

本题代码如下：

```
LinkList DisCreat_2(LinkList &A){
    LinkList B=(LinkList)malloc(sizeof(LNode));//创建B表表头
    B->next=NULL;                  //B表的初始化
    LNode *p=A->next,*q;           //p为工作指针
    LNode *ra=A;                   //ra始终指向A的尾结点
    while(p!=NULL){
        ra->next=p; ra=p;          //将*p链到A的表尾
        p=p->next;
        if(p!=NULL){
            q=p->next;             //头插后，*p将断链，因此用q记忆*p的后继
            p->next=B->next;       //将*p插入到B的前端
            B->next=p;
            p=q;
        }
    }
    ra->next=NULL;                 //A尾结点的next域置空
    return B;
}
```

该算法特别需要注意的是，采用头插法插入结点后，*p的指针域已改变，若不设变量保存其后继结点，则会引起断链，从而导致算法出错。

07.【解答】

算法思想：题中链表是有序表，因此所有相同值域的结点都是相邻的。用p扫描递增单链表L，若*p结点的值域等于其后继结点的值域，则删除后者，否则p移向下一个结点。

本题代码如下：

```
void Del_Same(LinkList &L){
    LNode *p=L->next,*q;           //p为扫描工作指针
    if(p==NULL)
        return;
    while(p->next!=NULL){
        q=p->next;                 //q指向*p的后继结点
        if(p->data==q->data){      //找到重复值的结点
            p->next=q->next;       //释放*q结点
            free(q);               //释放相同元素值的结点
        }
        else
            p=p->next;
    }
}
```

本算法的时间复杂度为$O(n)$，空间复杂度为$O(1)$。

本题也可采用尾插法，将头结点摘下，然后从第一结点开始，依次与已经插入结点的链表的

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

最后一个结点比较，若不等则直接插入，否则将当前遍历的结点删除并处理下一个结点，直到最后一个结点为止。

08.【解答】

算法思想：表 A、B 都有序，可从第一个元素起依次比较 A、B 两表的元素，若元素值不等，则值小的指针往后移，若元素值相等，则创建一个值等于两结点的元素值的新结点，使用尾插法插入到新的链表中，并将两个原表指针后移一位，直到其中一个链表遍历到表尾。

本题代码如下：

```
void Get_Common(LinkList A,LinkList B){
    LNode *p=A->next,*q=B->next,*r,*s;
    LinkList C=(LinkList)malloc(sizeof(LNode));  //建立表 C
    r=C;                                         //r 始终指向 C 的尾结点
    while(p!=NULL&&q!=NULL){                     //循环跳出条件
        if(p->data<q->data)
            p=p->next;                           //若 A 的当前元素较小，后移指针
        else if(p->data>q->data)
            q=q->next;                           //若 B 的当前元素较小，后移指针
        else{                                    //找到公共元素结点
            s=(LNode*)malloc(sizeof(LNode));
            s->data=p->data;                     //复制产生结点*s
            r->next=s;                           //将*s 链接到 C 上（尾插法）
             r=s;
            p=p->next;                           //表 A 和 B 继续向后扫描
            q=q->next;
        }
    }
    r->next=NULL;                                //置 C 尾结点指针为空
}
```

09.【解答】

算法思想：采用归并的思想，设置两个工作指针 pa 和 pb，对两个链表进行归并扫描，只有同时出现在两集合中的元素才链接到结果表中且仅保留一个，其他的结点全部释放。当一个链表遍历完毕后，释放另一个表中剩下的全部结点。

本题代码如下：

```
LinkList Union(LinkList &la,LinkList &lb){
    LNode *pa=la->next;                  //设工作指针分别为 pa 和 pb
    LNode *pb=lb->next;
    LNode *u,*pc=la;                     //结果表中当前合并结点的前驱指针 pc
    while(pa&&pb){
        if(pa->data==pb->data){          //交集并入结果表中
            pc->next=pa;                 //A 中结点链接到结果表
            pc=pa;
            pa=pa->next;
            u=pb;                        //B 中结点释放
            pb=pb->next;
            free(u);
        }
        else if(pa->data<pb->data){      //若 A 中当前结点值小于 B 中当前结点值
            u=pa;
            pa=pa->next;                 //后移指针
            free(u);                     //释放 A 中当前结点
        }
        else{                            //若 B 中当前结点值小于 A 中当前结点值
            u=pb;
```

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

```
            pb=pb->next;                    //后移指针
            free(u);                        //释放B中当前结点
        }
    }//while结束
    while(pa){                              //B已遍历完，A未完
        u=pa;
        pa=pa->next;
        free(u);                            //释放A中剩余结点
    }
    while(pb){                              //A已遍历完，B未完
        u=pb;
        pb=pb->next;
        free(u);                            //释放B中剩余结点
    }
    pc->next=NULL;                          //置结果链表尾指针为NULL
    free(lb);                               //释放B表的头结点
    return  la;
}
```

链表归并类型的试题在各学校历年真题中出现的频率很高，故应扎实掌握解决此类问题的思想。该算法的时间复杂度为$O(\text{len1}+\text{len2})$，空间复杂度为$O(1)$。

10.【解答】

算法思想：因为两个整数序列已存入两个链表中，操作从两个链表的第一个结点开始，若对应数据相等，则后移指针；若对应数据不等，则A链表从上次开始比较结点的后继开始，B链表仍从第一个结点开始比较，直到B链表到尾表示匹配成功。A链表到尾而B链表未到尾表示失败。操作中应记住A链表每次的开始结点，以便下次匹配时好从其后继开始。

本题代码如下：

```
int Pattern(LinkList A,LinkList B){
    LNode *p=A;                 //p为A链表的工作指针，本题假定A和B均无头结点
    LNode *pre=p;               //pre记住每趟比较中A链表的开始结点
    LNode *q=B;                 //q是B链表的工作指针
    while(p&&q)
        if(p->data==q->data){//结点值相同
            p=p->next;
            q=q->next;
        }
        else{
            pre=pre->next;
            p=pre;              //A链表新的开始比较结点
            q=B;                //q从B链表第一个结点开始
        }
    if(q==NULL)                 //B已经比较结束
        return  1;              //说明B是A的子序列
    else
        return  0;              //B不是A的子序列
}
```

注　意

该题其实是字符串模式匹配的链式表示形式，读者应该结合字符串模式匹配的内容重新考虑能否优化该算法。

11.【解答】

算法思想：让p从左向右扫描，q从右向左扫描，直到它们指向同一结点（p==q，当循环双链

配套课程公众号[考机研]
永久联系微信:4550060

表中结点个数为奇数时）或相邻（p->next=q 或 q->prior=p，当循环双链表中结点个数为偶数时）为止，若它们所指结点值相同，则继续进行下去，否则返回 0。若比较全部相等，则返回 1。

本题代码如下：

```
int Symmetry(DLinkList L){
    DNode *p=L->next,*q=L->prior;   //两头工作指针
    while(p!=q&&p->next!=q)         //循环跳出条件
        if(p->data==q->data){       //所指结点值相同则继续比较
            p=p->next;
            q=q->prior;
         }
        else                        //否则，返回 0
            return  0;
    return  1;                      //比较结束后返回 1
}
```

注　意

while 循环第二个判断条件易误写成 q->next!=p，分析这样会产生什么问题。

12.【解答】

算法思想：先找到两个链表的尾指针，将第一个链表的尾指针与第二个链表的头结点链接起来，再使之成为循环的。

本题代码如下：

```
LinkList Link(LinkList &h1,LinkList &h2){
//将循环链表 h2 链接到循环链表 h1 之后，使之仍保持循环链表的形式
    LNode *p,*q;                    //分别指向两个链表的尾结点
    p=h1;
    while(p->next!=h1)              //寻找 h1 的尾结点
        p=p->next;
    q=h2;
    while(q->next!=h2)              //寻找 h2 的尾结点
        q=q->next;
    p->next=h2;                     //将 h2 链接到 h1 之后
    q->next=h1;                     //令 h2 的尾结点指向 h1
    return  h1;
}
```

13.【解答】

算法思想：首先在双链表中查找数据值为 x 的结点，查到后，将结点从链表上摘下，然后顺着结点的前驱链查找该结点的插入位置（频度递减，且排在同频度的第一个，即向前找到第一个比它的频度大的结点，插入位置为该结点之后），并插入到该位置。

本题代码如下：

```
DLinkList Locate(DLinkList &L,ElemType x){
    DNode *p=L->next,*q;            //p 为工作指针，q 为 p 的前驱，用于查找插入位置
    while(p&&p->data!=x)
        p=p->next;                  //查找值为 x 的结点
    if(!p)
        exit(0);                    //不存在值为 x 的结点
    else{
        p->freq++;                  //令元素值为 x 的结点的 freq 域加 1
        if(p->pre==L||p->pre->freq>p->freq)
            return  p;              //p 是链表首结点，或 freq 值小于前驱
        if(p->next!=NULL) p->next->pre=p->pre;
```

配套课程公众号[考机研]
永久联系微信:4550060

```
            p->pre->next=p->next;    //将 p 结点从链表上摘下
            q=p->pre;                //以下查找 p 结点的插入位置
            while(q!=L&&q->freq<=p->freq)
                q=q->pre;
            p->next=q->next;
            if(q->next!=NULL) q->next->pre=p;   //将 p 结点排在同频率的第一个
            p->pre=q;
            q->next=p;
        }
        return  p;                   //返回值为 x 的结点的指针
    }
```

14.【解答】

1）算法的基本设计思想：

首先，遍历链表计算表长 n，并找到链表的尾结点，将其与首结点相连，得到一个循环单链表。然后，找到新链表的尾结点，它为原链表的第 $n-k$ 个结点，令 L 指向新链表尾结点的下一个结点，并将环断开，得到新链表。

2）本题代码如下：

```
LNode *Converse(LNode *L,int k){
    int n=1;                     //n 用来保存链表的长度
    LNode *p=L;                  //p 为工作指针
    while(p->next!=NULL){        //计算链表的长度
        p=p->next;
        n++;
    }                            //循环执行完后，p 指向链表尾结点
    p->next=L;                   //将链表连成一个环
    for(int i=1;i<=n-k;i++)      //寻找链表的第 n-k 个结点
        p=p->next;
    L=p->next;                   //令 L 指向新链表尾结点的下一个结点
    p->next=NULL;                //将环断开
    return L;
}
```

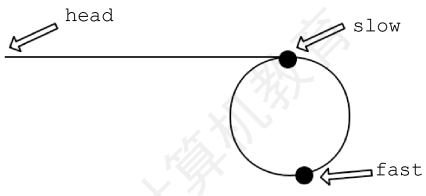
3）本算法的时间复杂度为 $O(n)$，空间复杂度为 $O(1)$。

15.【解答】

1）算法的基本设计思想：

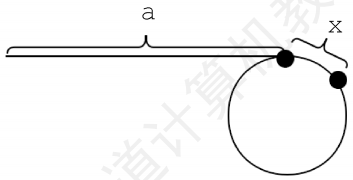
设置快慢两个指针分别为 fast 和 slow 最初都指向链表头 head。slow 每次走一步，即 slow=slow->next；fast 每次走两步，即 fast=fast->next->next。fast 比 slow 走得快，若有环，则 fast 一定先进入环，而 slow 后进入环。两个指针都进入环后，经过若干操作后两个指针定能在环上相遇。这样就可以判断一个链表是否有环。

如下图所示，当 slow 刚进入环时，fast 早已进入环。因为 fast 每次比 slow 多走一步且 fast 与 slow 的距离小于环的长度，所以 fast 与 slow 相遇时，slow 所走的距离不超过环的长度。



如下图所示，设头结点到环的入口点的距离为 a，环的入口点沿着环的方向到相遇点的距离为 x，环长为 r，相遇时 fast 绕过了 n 圈。

配套课程公众号[考机研]
永久联系微信:4550060



则有 2(a+x)=a+n*r+x，即 a=nr－x。显然从头结点到环的入口点的距离等于 n 倍的环长减去环的入口点到相遇点的距离。因此可设置两个指针，一个指向 head，一个指向相遇点，两个指针同步移动（均为一次走一步），相遇点即环的入口点。

2）本题代码如下：

```
LNode* FindLoopStart(LNode *head){
    LNode *fast=head, *slow=head;     //设置快慢两个指针
    while(fast!=NULL&&fast->next!=NULL){
        slow=slow->next;              //每次走一步
        fast=fast->next->next;        //每次走两步
        if(slow==fast)  break;        //相遇
    }
    if(fast==NULL||fast->next==NULL)
        return NULL;                  //没有环，返回 NULL
    LNode *p1=head, *p2=slow ;        //分别指向开始点、相遇点
    while(p1!=p2){
        p1=p1->next;
        p2=p2->next;
    }
    return p1;                        //返回入口点
}
```

3）当 fast 与 slow 相遇时，slow 肯定没有遍历完链表，故算法的时间复杂度为 $O(n)$，空间复杂度为 $O(1)$。

16.【解答】

1）算法的基本设计思想：

设置快、慢两个指针分别为 fast 和 slow，初始时 slow 指向 L（第一个结点），fast 指向 L->next（第二个结点），之后 slow 每次走一步，fast 每次走两步。当 fast 指向表尾（第 n 个结点）时，slow 正好指向链表的中间点（第 $n/2$ 个结点），即 slow 正好指向链表前半部分的最后一个结点。将链表的后半部分逆置，然后设置两个指针分别指向链表前半部分和后半部分的首结点，在遍历过程中计算两个指针所指结点的元素之和，并维护最大值。

2）本题代码如下：

```
int PairSum(LinkList L){
    LNode *fast=L->next,*slow=L;//利用快慢双指针找到链表的中间点
    while(fast!=NULL&&fast->next!=NULL){
        fast=fast->next->next;    //快指针每次走两步
        slow=slow->next;          //慢指针每次走一步
    }
    LNode *newHead=NULL,*p=slow->next,*tmp;
    while(p!=NULL){               //反转链表后一半部分的元素，采用头插法
        tmp=p->next;              //p 指向当前待插入结点，令 tmp 指向其下一结点
        p->next=newHead;          //将 p 所指结点插入到新链表的首结点之前
        newHead=p;                //newHead 指向刚才新插入的结点，作为新的首结点
        p=tmp;                    //当前待处理结点变为下一结点
    }
    int mx=0; p=L;
    LNode *q=newHead;
```

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

```
    while(q!=NULL){                    //用 p 和 q 分别遍历两个链表
        if((p->data+q->data)>mx) //用 mx 记录最大值
            mx=p->data+q->data;
        p=p->next;
        q=q->next;
    }
    return mx;
}
```

3）本算法的时间复杂度为 $O(n)$，空间复杂度为 $O(1)$。

17.【解答】

1）算法的基本设计思想：

问题的关键是设计一个尽可能高效的算法，通过链表的一次遍历，找到倒数第 k 个结点的位置。定义两个指针变量 p 和 q，初始时均指向头结点的下一个结点（链表的第一个结点），p 指针沿链表移动；当 p 指针移动到第 k 个结点时，q 指针开始与 p 指针同步移动；当 p 指针移动到最后一个结点时，q 指针所指示结点为倒数第 k 个结点。以上过程对链表仅进行一遍扫描。

2）算法的详细实现步骤如下：

① count=0，p 和 q 指向链表表头结点的下一个结点。

② 若 p 为空，转⑤。

③ 若 count 等于 k，则 q 指向下一个结点；否则，count=count+1。

④ p 指向下一个结点，转②。

⑤ 若 count 等于 k，则查找成功，输出该结点的 data 域的值，返回 1；否则，说明 k 值超过了线性表的长度，查找失败，返回 0。

⑥ 算法结束。

3）算法实现如下：

```
typedef int ElemType;                  //链表数据的类型定义
typedef struct LNode{                  //链表结点的结构定义
    ElemType  data;                    //结点数据
    struct LNode *link;                //结点链接指针
}LNode, *LinkList;
int Search_k(LinkList  list,int  k){
    LNode *p=list->link,*q=list->link; //指针 p、q 指示第一个结点
    int count=0;
    while(p!=NULL){                    //遍历链表直到最后一个结点
        if (count<k) count++;          //计数，若 count<k 只移动 p
        else q=q->link;
        p=p->link;                     //之后让 p、q 同步移动
    }//while
    if(count<k)
        return  0;                     //查找失败返回 0
    else {                             //否则打印并返回 1
    printf("%d",q->data);
    return  1;
    }
}
```

评分说明

若所给算法采用一遍扫描方式就能得到正确结果，则可给满分 15 分；若采用两遍或多遍扫描才能得到正确结果，则最高分为 10 分。若采用递归算法得到正确结果，则最高给 10 分；若实现算法的空间复杂度过高（使用了大小与 k 有关的辅助数组），但结果正确，则最高给 10 分。

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

18.【解答】

顺序遍历两个链表到尾结点时，并不能保证两个链表同时到达尾结点。这是因为两个链表的长度不同。假设一个链表比另一个链表长 k 个结点，我们先在长链表上遍历 k 个结点，之后同步遍历两个链表，这样就能够保证它们同时到达最后一个结点。因为两个链表从第一个公共结点到链表的尾结点都是重合的，所以它们肯定同时到达第一个公共结点。

1）算法的基本设计思想：

① 分别求出 str1 和 str2 所指的两个链表的长度 m 和 n。

② 将两个链表以表尾对齐：令指针 p、q 分别指向 str1 和 str2 的头结点，若 $m \geqslant n$，则指针 p 先走，使 p 指向链表中的第 $m-n+1$ 个结点；若 $m < n$，则使 q 指向链表中的第 $n-m+1$ 个结点，即使指针 p 和 q 所指的结点到表尾的长度相等。

③ 反复将指针 p 和 q 同步向后移动，并判断它们是否指向同一结点。当 p、q 指向同一结点，则该点即所求的共同后缀的起始位置。

2）本题代码如下：

```
typedef struct Node{
    char data;
    struct Node *next;
}SNode;
/*求链表长度的函数*/
int listlen(SNode *head){
    int len=0;
    while(head->next!=NULL){
        len++;
        head=head->next;
    }
    return len;
}
/*找出共同后缀的起始地址*/
SNode* find_list(SNode *str1,SNode *str2){
    int m,n;
    SNode *p,*q;
    m=listlen(str1);          //求 str1 的长度，O(m)
    n=listlen(str2);          //求 str2 的长度，O(n)
    for(p=str1;m>n;m--)       //若 m>n，使 p 指向链表中的第 m-n+1 个结点
        p=p->next;
    for(q=str2;m<n;n--)       //若 m<n，使 q 指向链表中的第 n-m+1 个结点
        q=q->next;
    while(p->next!=NULL&&p->next!=q->next){ //查找共同后缀起始点
        p=p->next;            //两个指针同步向后移动
        q=q->next;
    }
    return p->next;       //返回共同后缀的起始地址
}
```

3）时间复杂度为 $O(\text{len1}+\text{len2})$或 $O(\max(\text{len1}, \text{len2}))$，其中 len1、len2 分别为两个链表的长度。

19.【解答】

1）算法的基本设计思想：

- 算法的核心思想是用空间换时间。使用辅助数组记录链表中已出现的数值，从而只需对链表进行一趟扫描。
- 因为|data|≤n，故辅助数组 q 的大小为 $n+1$，各元素的初值均为 0。依次扫描链表中的各结点，同时检查 q[|data|]的值，若为 0 则保留该结点，并令 q[|data|]=1；否

配套课程公众号[考机研]
永久联系微信:4550060

则将该结点从链表中删除。

2）使用C语言描述的单链表结点的数据类型定义：

```
typedef struct node {
   int           data;
   struct node   *link;
}NODE;
Typedef NODE *PNODE;
```

3）算法实现如下：

```
void func (PNODE h,int n){
    PNODE p=h,r;
    int *q,m;
    q=(int *)malloc(sizeof(int)*(n+1));  //申请n+1个位置的辅助空间
    for(int i=0;i<n+1;i++)               //数组元素初值置0
       *(q+i)=0;
    while(p->link!=NULL){
       m=p->link->data>0? p->link->data:-p->link->data;
       if(*(q+m)==0){                    //判断该结点的data是否已出现过
          *(q+m)=1;                      //首次出现
          p=p->link;                     //保留
       }
       else{                             //重复出现
          r=p->link;                     //删除
          p->link=r->link;
          free(r);
       }
    }
    free(q);
}
```

4）参考答案所给算法的时间复杂度为$O(m)$，空间复杂度为$O(n)$。

20.【解答】

1）算法的基本设计思想：

先观察$L=(a_1,a_2,a_3,\cdots,a_{n-2},a_{n-1},a_n)$和$L'=(a_1,a_n,a_2,a_{n-1},a_3,a_{n-2},\cdots)$，发现$L'$是由$L$摘取第一个元素，再摘取倒数第一个元素……依次合并而成的。为了方便链表后半段取元素，需要先将L后半段原地逆置［题目要求空间复杂度为$O(1)$，不能借助栈］，否则每取最后一个结点都需要遍历一次链表。①先找出链表L的中间结点，为此设置两个指针p和q，指针p每次走一步，指针q每次走两步，当指针q到达链尾时，指针p正好在链表的中间结点；②然后将L的后半段结点原地逆置。③从单链表前后两段中依次各取一个结点，按要求重排。

2）算法实现如下：

```
void change_list(NODE*h){
   NODE *p,*q,*r,*s;
   p=q=h;
   while(q->next!=NULL){                 //寻找中间结点
      p=p->next;                         //p走一步
      q=q->next;
      if(q->next!=NULL) q=q->next;       //q走两步
   }
   q=p->next;          //p所指结点为中间结点，q为后半段链表的首结点
   p->next=NULL;
   while(q!=NULL){     //将链表后半段逆置
      r=q->next;
      q->next=p->next;
      p->next=q;
      q=r;
```

王道计算机教育 www.cskaoyan.com
配套课程公众号[考机研]
永久联系微信:4550060

```
        }
        s=h->next;              //s 指向前半段的第一个数据结点，即插入点
        q=p->next;              //q 指向后半段的第一个数据结点
        p->next=NULL;
        while(q!=NULL){         //将链表后半段的结点插入到指定位置
            r=q->next;          //r 指向后半段的下一个结点
            q->next=s->next;    //将 q 所指结点插入到 s 所指结点之后
            s->next=q;
            s=q->next;          //s 指向前半段的下一个插入点
            q=r;
        }
    }
```

3）第一步找中间结点的时间复杂度为 $O(n)$，第二步逆置的时间复杂度为 $O(n)$，第三步合并链表的时间复杂度为 $O(n)$，所以该算法的时间复杂度为 $O(n)$。

归纳总结

本章是算法设计题的重点考查章节，因为线性表的算法题的代码量一般都比较少，又具有一定的算法设计技巧，因此适合笔试考查。考研题中常以三段式的结构命题。

在给出题目背景和要求的情况下：

① 给出算法的基本设计思想。

② 采用 C 或 C++语言描述算法，并给出注释。

③ 分析所设计算法的时间复杂度和空间复杂度。

算法具体的设计思想千变万化，难以从一而定。因此读者一定要勤加练习，反复咀嚼本章的练习题，采用多种方法进行设计并比较它们的复杂度，逐渐熟悉各类题型的思考角度和最佳思路。这里，编者列出几种常用的算法设计技巧，仅供参考：对于链表，经常采用的方法有头插法、尾插法、逆置法、归并法、双指针法等，对具体问题需要灵活变通；对于顺序表，因为可以直接存取，所以经常结合排序和查找的几种算法设计思路进行设计，如归并排序、二分查找等。

注　意

对于算法设计题，若能写出数据结构类型的定义、正确的算法思想，则至少会给一半的分数；若能用伪代码写出，则自然更好；比较复杂的地方可以直接用文字表达。

思维拓展

一个长度为 n 的整型数组 A[1…n]，给定整数 x，设计一个时间复杂度不超过 $O(n\log_2 n)$的算法，查找出这个数组中所有两两之和等于 x 的整数对（每个元素只输出一次）。

提示：本题若想到排序，则问题便迎刃而解。先用一种时间复杂度为 $O(n\log_2 n)$的排序算法将 A[1…n]从小到大排序，然后分别从数组的小端(i=1)和大端(j=n)开始查找：若 A[i]+A[j]<x，i++；若 A[i]+A[j]>x，j--；否则输出 A[i]、A[j]，然后 i++，j--；直到 i>=j 时停止。

请读者思考本题是否有其他求解算法。

配套课程公众号[考机研]
永久联系微信:4550060